



UNIVERSITÀ
DEGLI STUDI
DI TRIESTE

UNIVERSITY OF TRIESTE

MASTER IN DATA SCIENCE AND SCIENTIFIC COMPUTING
CURRICULUM OF DATA SCIENCE AND APPLICATIONS

Course of Data Management for Big Data

Benchmark for Decision Support: TPC-H Implementation and Query Optimization

Authors:

Manasa Santhoshi Maddi (SM3500576)

Safa Yassin (SM3500583)

Department of Mathematics, Computer Science and Geosciences
2025

Contents

Contents	1
Introduction	3
1 Database and Schema Creation	4
1.1 Database Creation and Connection	4
1.2 SQL Definition of the Tables and Constraints	5
1.2.1 Creating the REGION Table	5
1.2.2 Creating the NATION Table	5
1.2.3 Creating the PART Table	6
1.2.4 Creating the SUPPLIER Table	7
1.2.5 Creating the PARTSUPP Table	7
1.2.6 Creating the CUSTOMER Table	8
1.2.7 Creating the ORDERS Table	9
1.2.8 Creating the LINEITEM Table	10
2 Data Population and Statistical Analysis	12
2.1 Data Population	12
2.2 Data Generation	12
2.3 Data Conversion from .tbl to .csv	13
2.4 Populating the DB using data generated by dbgen CSV files	14
3 Statistics of the Data	16
3.1 Size of the Database	16
3.2 Number of Rows in Each Table	16
3.3 Table Sizes in Bytes	17
3.4 Distinct Values for Relevant Attributes	19
3.5 Minimum and Maximum Values for Relevant Attributes	19
4 Query Schema and Analysis Before Optimization	21
4.1 Pricing Summary Report Query (Q1)	21
4.2 Shipping Priority Query (Q3)	22
4.3 Local Supplier Volume Query (Q5)	23
4.4 Volume Shipping Query (Q7)	25
4.5 Top Supplier Query (Q15)	27
4.6 Global Sales Opportunity Query (Q22)	29
5 Query Optimization	32

5.1	Optimization Strategy Overview	32
5.2	Optimization:Discussion and Strategy	33
5.2.1	Query 1: Pricing Summary Report	35
5.2.2	Query 3: Shipping Priority	36
5.2.3	Query 5: Local Supplier Volume	37
5.2.4	Query 7: Volume Shipping by Nation	38
5.2.5	Query 15: Top Supplier	40
5.2.6	Query 22: Global Sales Opportunity	41
5.3	Overall Performance Impact	43
5.3.1	Summary of Optimization Structures	43
6	Conclusion	44

Introduction

The present work focuses on the implementation and optimization of the TPC-H benchmark using PostgreSQL for decision support systems. The TPC-H benchmark is a standard used to evaluate the performance of database systems through a suite of business-oriented ad-hoc queries and concurrent data modifications, designed to simulate decision support systems. the related documentation can be found here: https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf

The primary objective of this project was to optimize the performance of six chosen queries, by exploring and leveraging both indexing techniques and materialized views. We first create a detailed database schema as per the TPC-H specifications, followed by data population using the TPC-H dbgen tool at a scale factor of 10, resulting in a **15GB** dataset. An initial statistical analysis was conducted to understand the data distribution and characteristics.

The optimization phase involved the strategic creation of indexes on key columns to reduce search times and improve join operations. Materialized views were designed to store precomputed query results, significantly enhancing query performance. Space constraints for indexes and materialized views were carefully verified to ensure compliance with specified limits.

Chapter 1

Database and Schema Creation

The first step in this project was to design the database schema according to the TPC-H specifications. The schema consists of several tables, each with its own structure, constraints, and data types. The process started with setting up the database and establishing a successful connection.

From there, tables such as REGION, NATION, PART, SUPPLIER, PARTSUPP, CUSTOMER, ORDERS, and LINEITEM were carefully created. Each table was defined with appropriate primary and foreign key constraints, as well as integrity checks to maintain data consistency. Screenshots were provided to demonstrate the successful creation and population of these tables.

1.1 Database Creation and Connection

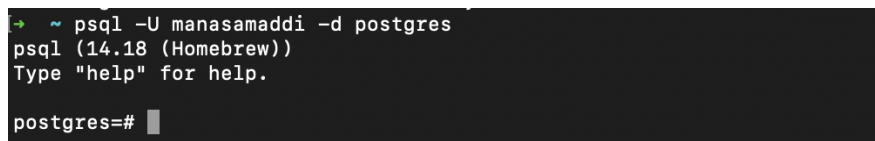
Here are the details about the database creation along with a screenshot of the connection.

- **Database Name:** Postgres
- **Database Type:** PostgreSQL
- **Owner:** manasamaddi
- **Port:** 5432

To connect to the database, I used the following command:

```
psql -U manasamaddi -d postgres
```

The screenshot below (Figure 1.1) shows the successful connection to the database:

A terminal window with a dark background. The prompt is a green arrow followed by a tilde. The command 'psql -U manasamaddi -d postgres' has been entered. The output shows 'psql (14.18 (Homebrew))' and 'Type "help" for help.' The prompt has changed to 'postgres=#' followed by a cursor.

```
➔ ~ psql -U manasamaddi -d postgres
psql (14.18 (Homebrew))
Type "help" for help.

postgres=#
```

Figure 1.1: Connecting to Postgres database

1.2 SQL Definition of the Tables and Constraints

The following section details the creation of each table in the PostgreSQL database, including integrity and check constraints. Screenshots of the table creation process are provided for each table. The tables were created and populated in a specific order to avoid integrity constraint violations.

1.2.1 Creating the REGION Table

The REGION table (Figure 1.2) is created first because it serves as a reference for the NATION table. This table includes a primary key constraint on R_REGIONKEY and a check constraint to ensure the key is non-negative.

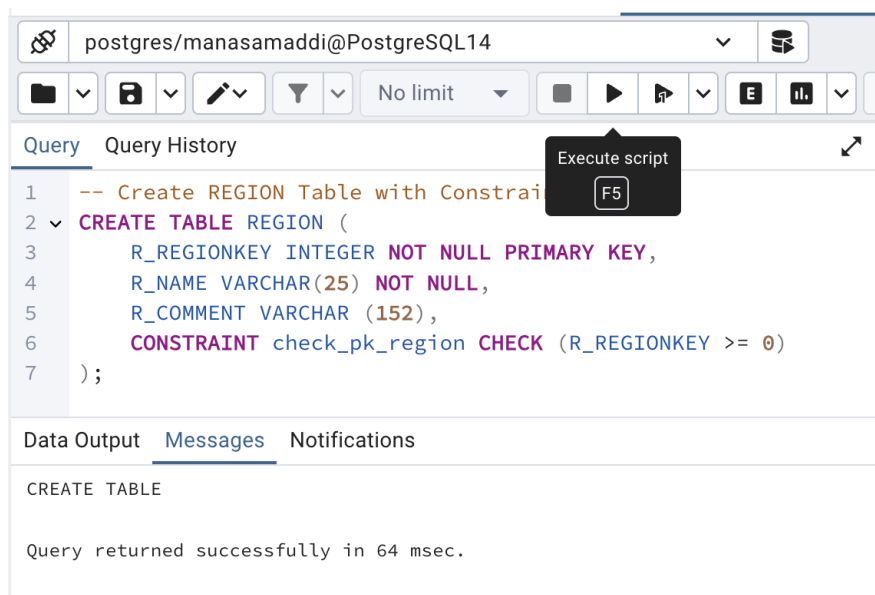


Figure 1.2: Creation of REGION Table

1.2.2 Creating the NATION Table

The NATION table (Figure 1.3) includes a primary key constraint on N_NATIONKEY, a foreign key constraint referencing REGION.R_REGIONKEY, and a check constraint to ensure the key is non-negative.

The screenshot shows a PostgreSQL query editor interface. At the top, the connection is 'postgres/manasamaddi@PostgreSQL14'. Below the toolbar, the 'Query' tab is active, displaying a SQL script to create a table named 'NATION'. The script includes a primary key constraint on 'N_NATIONKEY', a foreign key constraint on 'N_REGIONKEY' referencing 'REGION(R_REGIONKEY)', and a check constraint 'check_pk_nation' ensuring 'N_NATIONKEY' is greater than or equal to 0. The 'Data Output' tab is selected, showing the message 'CREATE TABLE' and a confirmation that the query was successful in 32 milliseconds.

```
1  -- create Nation Table with constraints
2  CREATE TABLE NATION (
3      N_NATIONKEY INTEGER NOT NULL PRIMARY KEY,
4      N_NAME VARCHAR(25) NOT NULL,
5      N_REGIONKEY INTEGER NOT NULL,
6      N_COMMENT VARCHAR(152),
7      FOREIGN KEY (N_REGIONKEY) REFERENCES REGION(R_REGIONKEY),
8      CONSTRAINT check_pk_nation CHECK (N_NATIONKEY >= 0)
9  );
```

CREATE TABLE

Query returned successfully in 32 msec.

Figure 1.3: Creation of NATION Table

1.2.3 Creating the PART Table

The PART table (Figure 1.4) includes a primary key (PK) constraint on P_PARTKEY and check constraints to ensure P_SIZE and P_RETAILPRICE are non-negative.

The screenshot shows a PostgreSQL query editor interface. At the top, the connection is 'postgres/manasamaddi@PostgreSQL14'. Below the toolbar, the 'Query' tab is active, displaying a SQL script to create a table named 'PART'. The script includes a primary key constraint on 'P_PARTKEY', several other columns with their data types, and two check constraints: 'check_pk_part' ensuring 'P_PARTKEY' is greater than or equal to 0, and another check constraint ensuring 'P_SIZE' is greater than or equal to 0. The 'Data Output' tab is selected, showing the message 'CREATE TABLE' and a confirmation that the query was successful in 39 milliseconds.

```
1  -- create Part Table with constraints
2  CREATE TABLE PART(
3      P_PARTKEY INTEGER NOT NULL PRIMARY KEY,
4      P_NAME VARCHAR(55),
5      P_MFGR CHAR(25),
6      P_BRAND CHAR(10),
7      P_TYPE VARCHAR(25),
8      P_SIZE INTEGER NOT NULL CHECK (P_SIZE >= 0),
9      P_CONTAINER CHAR(10),
10     P_RETAILPRICE DECIMAL NOT NULL CHECK (P_RETAILPRICE >= 0),
11     P_COMMENT VARCHAR(23),
12     CONSTRAINT check_pk_part CHECK (P_PARTKEY >= 0)
13 );
```

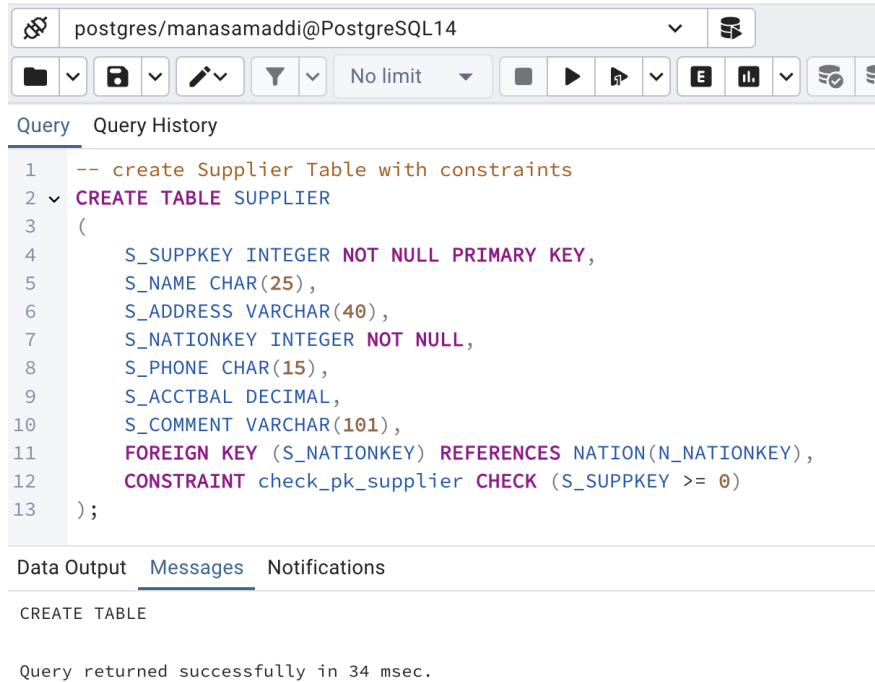
CREATE TABLE

Query returned successfully in 39 msec.

Figure 1.4: Creation of PART Table

1.2.4 Creating the SUPPLIER Table

The SUPPLIER table (Figure 1.5) includes a primary key (PK) constraint on S_SUPPKEY, a foreign key (FK) constraint referencing NATION.N_NATIONKEY, and a check constraint to ensure that the key is non-negative.

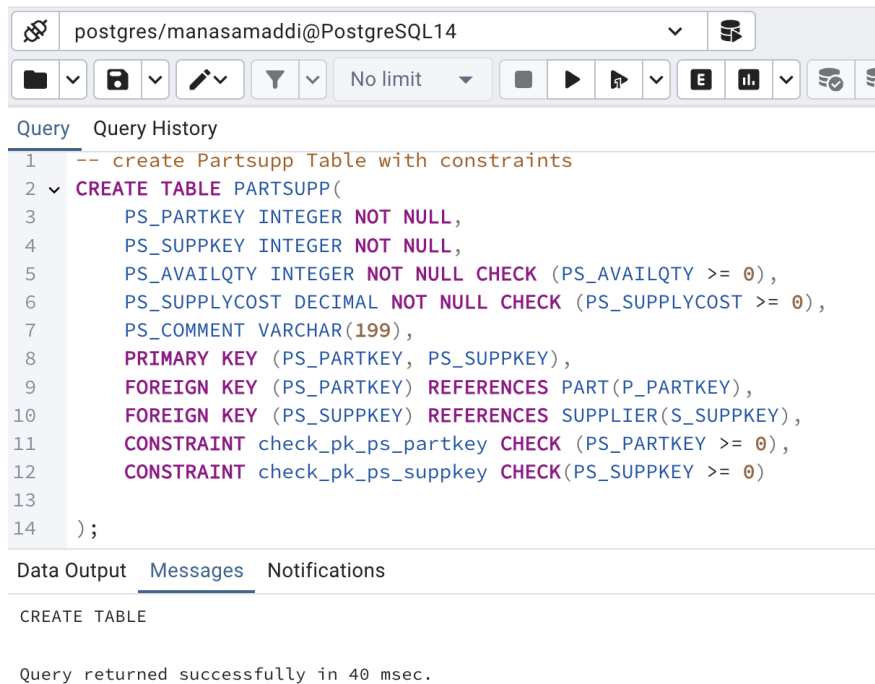


```
postgres/manasamaddi@PostgreSQL14
Query Query History
1 -- create Supplier Table with constraints
2 CREATE TABLE SUPPLIER
3 (
4     S_SUPPKEY INTEGER NOT NULL PRIMARY KEY,
5     S_NAME CHAR(25),
6     S_ADDRESS VARCHAR(40),
7     S_NATIONKEY INTEGER NOT NULL,
8     S_PHONE CHAR(15),
9     S_ACCTBAL DECIMAL,
10    S_COMMENT VARCHAR(101),
11    FOREIGN KEY (S_NATIONKEY) REFERENCES NATION(N_NATIONKEY),
12    CONSTRAINT check_pk_supplier CHECK (S_SUPPKEY >= 0)
13 );
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 34 msec.
```

Figure 1.5: Creation of SUPPLIER Table

1.2.5 Creating the PARTSUPP Table

The PARTSUPP table (Figure 1.6) includes a composite primary key (PK) constraint on PS_PARTKEY and PS_SUPPKEY, foreign key (FK) constraints referencing PART.P_PARTKEY and SUPPLIER.S_SUPPKEY, and check constraints to ensure that PS_AVAILQTY and PS_SUPPLYCOST are non-negative.



The screenshot shows a PostgreSQL query editor interface. At the top, the connection is 'postgres/manasamaddi@PostgreSQL14'. Below the connection bar is a toolbar with icons for file operations, query execution, and other database functions. The main area is divided into 'Query' and 'Query History' tabs. The 'Query' tab is active, displaying a SQL script to create the 'PARTSUPP' table. The script includes column definitions for 'PS_PARTKEY', 'PS_SUPPKEY', 'PS_AVAILQTY', 'PS_SUPPLYCOST', and 'PS_COMMENT', along with primary and foreign key constraints and check constraints. Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, showing the output 'CREATE TABLE' and a confirmation message: 'Query returned successfully in 40 msec.'

```
1  -- create Partsupp Table with constraints
2  CREATE TABLE PARTSUPP(
3      PS_PARTKEY INTEGER NOT NULL,
4      PS_SUPPKEY INTEGER NOT NULL,
5      PS_AVAILQTY INTEGER NOT NULL CHECK (PS_AVAILQTY >= 0),
6      PS_SUPPLYCOST DECIMAL NOT NULL CHECK (PS_SUPPLYCOST >= 0),
7      PS_COMMENT VARCHAR(199),
8      PRIMARY KEY (PS_PARTKEY, PS_SUPPKEY),
9      FOREIGN KEY (PS_PARTKEY) REFERENCES PART(P_PARTKEY),
10     FOREIGN KEY (PS_SUPPKEY) REFERENCES SUPPLIER(S_SUPPKEY),
11     CONSTRAINT check_pk_ps_partkey CHECK (PS_PARTKEY >= 0),
12     CONSTRAINT check_pk_ps_suppkey CHECK(PS_SUPPKEY >= 0)
13 )
14 ;
```

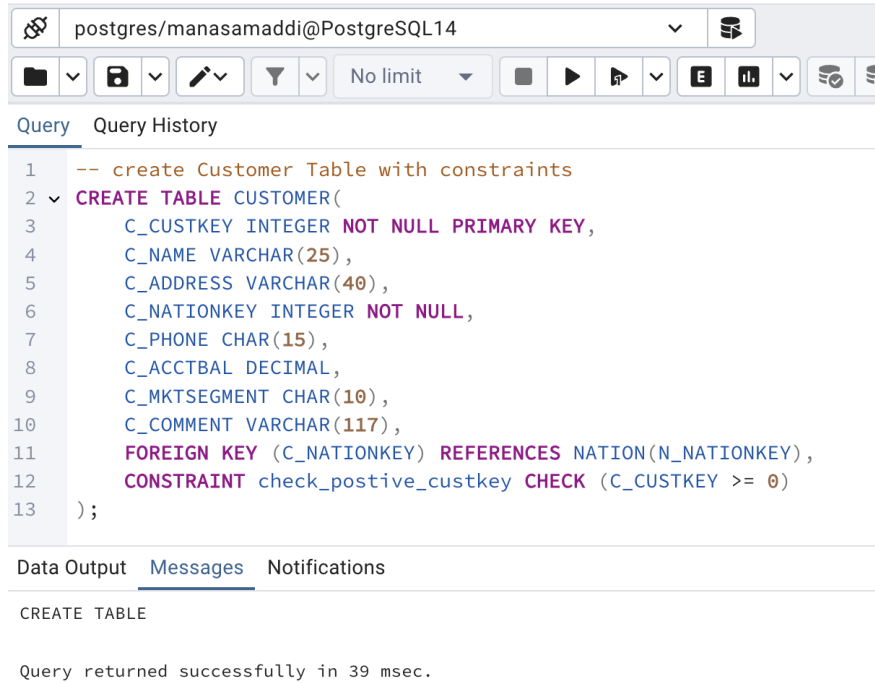
CREATE TABLE

Query returned successfully in 40 msec.

Figure 1.6: Creation of PARTSUPP Table

1.2.6 Creating the CUSTOMER Table

The CUSTOMER table (Figure 1.7) includes a primary key (PK) constraint on C_CUSTKEY, a foreign key (FK) constraint referencing NATION.N_NATIONKEY, and a check constraint to ensure the key is non-negative.



The screenshot shows a PostgreSQL query editor interface. At the top, the connection is identified as 'postgres/manasamaddi@PostgreSQL14'. Below the connection bar is a toolbar with icons for file operations, query execution, and other database functions. The main area displays a SQL query to create a 'CUSTOMER' table with various attributes and constraints. The query is as follows:

```
1  -- create Customer Table with constraints
2  CREATE TABLE CUSTOMER(
3      C_CUSTKEY INTEGER NOT NULL PRIMARY KEY,
4      C_NAME VARCHAR(25),
5      C_ADDRESS VARCHAR(40),
6      C_NATIONKEY INTEGER NOT NULL,
7      C_PHONE CHAR(15),
8      C_ACCTBAL DECIMAL,
9      C_MKTSEGMENT CHAR(10),
10     C_COMMENT VARCHAR(117),
11     FOREIGN KEY (C_NATIONKEY) REFERENCES NATION(N_NATIONKEY),
12     CONSTRAINT check_postive_custkey CHECK (C_CUSTKEY >= 0)
13 );
```

Below the query editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the output of the query: 'CREATE TABLE'. At the bottom, a status message indicates 'Query returned successfully in 39 msec.'

Figure 1.7: Creation of CUSTOMER Table

1.2.7 Creating the ORDERS Table

The `ORDERS` table (Figure 1.8) includes a primary key (PK) constraint on `O_ORDERKEY`, a foreign key (FK) constraint referencing `CUSTOMER.C_CUSTKEY`, and a check constraint to ensure that `O_TOTALPRICE` is non-negative.

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the user is 'postgres/manasamaddi@PostgreSQL14'. Below the bar are various icons for file operations, query execution, and settings. The main area displays a SQL query to create an 'ORDERS' table with the following specifications:

```

1  -- create Order Table with constraints
2  CREATE TABLE ORDERS(
3      O_ORDERKEY INTEGER NOT NULL PRIMARY KEY,
4      O_CUSTKEY INTEGER NOT NULL,
5      O_ORDERSTATUS CHAR(1),
6      O_TOTALPRICE DECIMAL NOT NULL CHECK (O_TOTALPRICE >= 0),
7      O_ORDERDATE DATE,
8      O_ORDERPRIORITY CHAR(15),
9      O_CLERK CHAR(15),
10     O_SHIPPRIORITY INTEGER,
11     O_COMMENT VARCHAR(79),
12     FOREIGN KEY (O_CUSTKEY) REFERENCES CUSTOMER(C_CUSTKEY),
13     CONSTRAINT check_pk_orderkey CHECK (O_ORDERKEY >= 0)
14 );

```

Below the query, the 'Data Output' tab shows the result: 'CREATE TABLE'. A message at the bottom states: 'Query returned successfully in 28 msec.'

Figure 1.8: Creation of ORDERS Table

1.2.8 Creating the LINEITEM Table

The LINEITEM table (Figure 1.9) includes a composite primary key (PK) constraint on L_ORDERKEY and L_LINENUMBER, foreign key (FK) constraints referencing ORDERS(O_ORDERKEY) and PARTSUPP(PS_PARTKEY, PS_SUPPKEY), and check constraints to ensure that L_QUANTITY, L_EXTENDEDPRICE, and L_TAX are non-negative, and that L_DISCOUNT is between 0 and 1.

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'postgres/manasamaddi@PostgreSQL14'. Below the toolbar, the 'Query' tab is active, displaying a SQL script to create the 'LINEITEM' table. The script includes various data types, constraints, and foreign key references. The 'Data Output' tab is selected, showing the command 'CREATE TABLE' and a message: 'Query returned successfully in 32 msec.'

```
1  -- create Lineitem Table with constraints
2  CREATE TABLE LINEITEM(
3      L_ORDERKEY INTEGER NOT NULL,
4      L_PARTKEY INTEGER NOT NULL,
5      L_SUPPKEY INTEGER NOT NULL,
6      L_LINENUMBER INTEGER NOT NULL,
7      L_QUANTITY DECIMAL NOT NULL CHECK (L_QUANTITY >=0),
8      L_EXTENDEDPRICE DECIMAL NOT NULL CHECK (L_EXTENDEDPRICE >=0),
9      L_DISCOUNT DECIMAL NOT NULL CHECK (L_DISCOUNT BETWEEN 0 AND 1),
10     L_TAX DECIMAL NOT NULL CHECK (L_TAX >=0),
11     L_RETURNFLAG CHAR(1),
12     L_LINestatus CHAR(1),
13     L_SHIPDATE DATE,
14     L_COMMITDATE DATE,
15     L_RECEIPTDATE DATE CHECK (L_SHIPDATE <= L_RECEIPTDATE),
16     L_SHIPINSTRUCT CHAR(25),
17     L_SHIPMODE CHAR(10),
18     L_COMMENT VARCHAR(44),
19     PRIMARY KEY (L_ORDERKEY, L_LINENUMBER),
20     FOREIGN KEY (L_ORDERKEY) REFERENCES ORDERS(O_ORDERKEY),
21     FOREIGN KEY (L_PARTKEY, L_SUPPKEY) REFERENCES PARTSUPP(PARTKEY, SUPPKEY)
22 );
```

CREATE TABLE

Query returned successfully in 32 msec.

Figure 1.9: Creation of LINEITEM Table

The screenshot shows the same PostgreSQL query editor interface. The 'Query' tab is active, displaying a SQL script to alter the 'LINEITEM' table by adding two foreign key constraints. The 'Data Output' tab is selected, showing the command 'ALTER TABLE' and a message: 'Query returned successfully in 38 msec.'

```
1  ALTER TABLE LINEITEM
2  ADD CONSTRAINT fk_lineitem_part FOREIGN KEY (L_PARTKEY) REFERENCES PART(P_PARTKEY);
3
4  ALTER TABLE LINEITEM
5  ADD CONSTRAINT fk_lineitem_supp FOREIGN KEY (L_SUPPKEY) REFERENCES SUPPLIER(S_SUPPKEY);
6
```

ALTER TABLE

Query returned successfully in 38 msec.

Figure 1.10: Alter of LINEITEM Table

Chapter 2

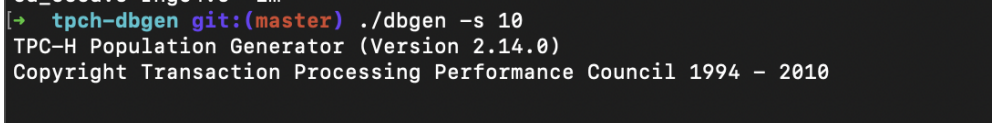
Data Population and Statistical Analysis

After defining the schema, the next step involved populating the database with data generated by the TPC-H `dbgen` tool at a Scale Factor (SF) of 10, ensuring a substantial dataset for performance testing and benchmarking.

This section details the process of generating the data, converting it into a suitable format for PostgreSQL, and executing the necessary commands to load the data into the database. Screenshots and command-line instructions illustrate each step from data generation to successful data loading, ensuring that the database is fully populated and ready for further analysis.

2.1 Data Population

After defining the schema, we populated the database using the data generated by the `dbgen` tool. **The data was generated with a Scale Factor (SF) of 10, as required.** Below are the details (Figure 2.1) of how we implemented the `dbgen` tool, generated the data, and populated the PostgreSQL database using the resulting CSV files.



```
[→ tpch-dbgen git:(master) ./dbgen -s 10
TPC-H Population Generator (Version 2.14.0)
Copyright Transaction Processing Performance Council 1994 - 2010
```

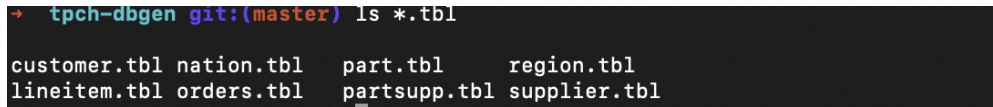
Figure 2.1: Data generation using the TPC-H tool

2.2 Data Generation

To generate data using the TPC-H tool, we executed the following command inside the `dbgen` folder:

```
./dbgen -s 10
```

This command generates the data with a Scale Factor (SF) of 10, which means the dataset is scaled to represent a larger volume of data suitable for performance testing and benchmarking. The generated data files are in the `.tbl` format, as shown in the Figure 2.2.



```
→ tpch-dbgen git:(master) ls *.tbl
customer.tbl nation.tbl part.tbl region.tbl
lineitem.tbl orders.tbl partsupp.tbl supplier.tbl
```

Figure 2.2: List of Tables in Data Generation

The `dbgen` tool generates several `.tbl` files corresponding to different tables in the TPC-H schema, such as `customer.tbl`, `lineitem.tbl`, `nation.tbl`, `orders.tbl`, `part.tbl`, `partsupp.tbl`, `region.tbl`, and `supplier.tbl`. These files contain the raw data that will be loaded into the PostgreSQL database.

The screenshot (Figure 2.2) shows the execution of the `dbgen` command and the resulting `.tbl` files that were generated. Each file corresponds to a table in the TPC-H schema and will be used to populate the database.

2.3 Data Conversion from `.tbl` to `.csv`

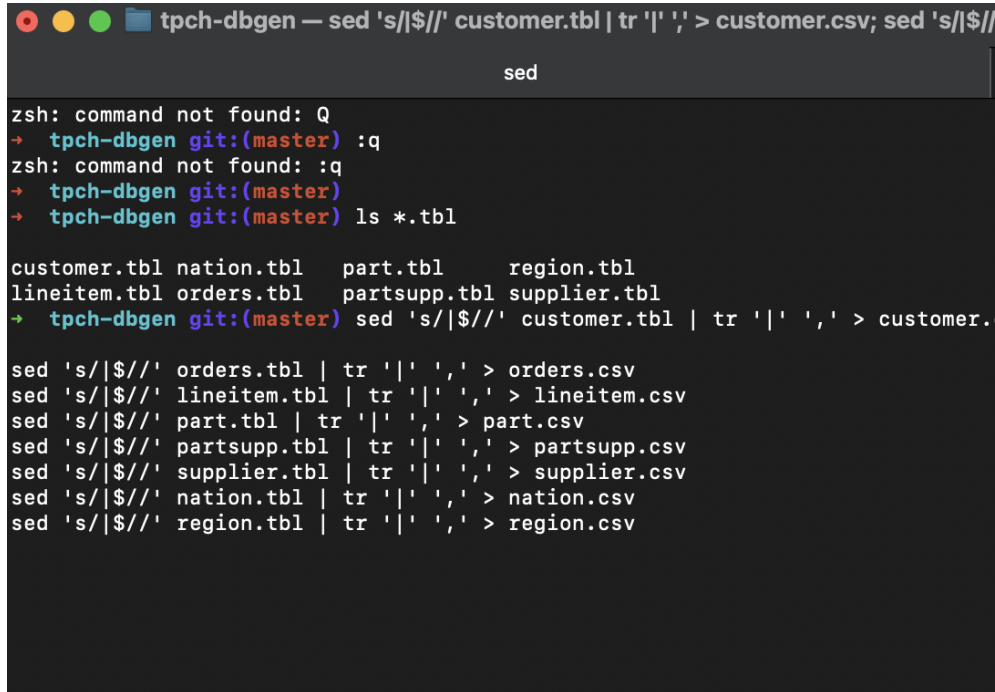
To convert the generated data files from the `.tbl` format to the `.csv` format, we used the following commands:

```
sed 's/|$//' customer.tbl | tr '|' ' ' > customer.csv
sed 's/|$//' orders.tbl | tr '|' ' ' > orders.csv
sed 's/|$//' lineitem.tbl | tr '|' ' ' > lineitem.csv
sed 's/|$//' part.tbl | tr '|' ' ' > part.csv
sed 's/|$//' partsupp.tbl | tr '|' ' ' > partsupp.csv
sed 's/|$//' supplier.tbl | tr '|' ' ' > supplier.csv
sed 's/|$//' nation.tbl | tr '|' ' ' > nation.csv
sed 's/|$//' region.tbl | tr '|' ' ' > region.csv
```

This command iterates through all the `.tbl` files, removes the trailing pipe character (`|`), and converts each file to a `.csv` file. The output files are then ready for loading into the PostgreSQL database.

The screenshot below (Figure 2.3) shows the execution of the command and the resulting `.csv` files:

The Figure 2.3 illustrates the execution of the terminal command and the resulting .csv files alongside the original .tbl files. Each .tbl file, such as customer.tbl, lineitem.tbl, nation.tbl, etc., has been successfully converted to its corresponding .csv file.



```
tpch-dbgen — sed 's/|$//' customer.tbl | tr '|' ',' > customer.csv; sed 's/|$//'  
sed  
zsh: command not found: Q  
→ tpch-dbgen git:(master) :q  
zsh: command not found: :q  
→ tpch-dbgen git:(master)  
→ tpch-dbgen git:(master) ls *.tbl  
  
customer.tbl nation.tbl part.tbl region.tbl  
lineitem.tbl orders.tbl partsupp.tbl supplier.tbl  
→ tpch-dbgen git:(master) sed 's/|$//' customer.tbl | tr '|' ',' > customer.  
  
sed 's/|$//' orders.tbl | tr '|' ',' > orders.csv  
sed 's/|$//' lineitem.tbl | tr '|' ',' > lineitem.csv  
sed 's/|$//' part.tbl | tr '|' ',' > part.csv  
sed 's/|$//' partsupp.tbl | tr '|' ',' > partsupp.csv  
sed 's/|$//' supplier.tbl | tr '|' ',' > supplier.csv  
sed 's/|$//' nation.tbl | tr '|' ',' > nation.csv  
sed 's/|$//' region.tbl | tr '|' ',' > region.csv
```

Figure 2.3: Conversion of data from .tbl to .csv format

2.4 Populating the DB using data generated by dbgen CSV files

To populate the data in a PostgreSQL database using CSV files, we used the following commands.

Note: It is important to load the data in the specified order to prevent violations of the integrity of the constraint.

Screenshots (Figures 2.4 & 2.5) of the command executions are shown below.

```

postgres=# COPY region FROM '/Users/manasamaddi/Desktop/DBMS/DATA_TABLES/region.csv' DELIMITER '|' CSV;
COPY 5
postgres=# COPY nation FROM '/Users/manasamaddi/Desktop/DBMS/DATA_TABLES/nation.csv' DELIMITER '|' CSV;
COPY 25
postgres=# COPY part FROM '/Users/mana ~ -- psql postgres -- psql -- psql postgres part.csv' DELIMITER '|' CSV;
COPY 2000000
postgres=# COPY supplier FROM '/Users/manasamaddi/Desktop/DBMS/DATA_TABLES/supplier.csv' DELIMITER '|' CSV;
COPY 100000
postgres=# COPY partsupp FROM '/Users/manasamaddi/Desktop/DBMS/DATA_TABLES/partsupp.csv' DELIMITER '|' CSV;
COPY 8000000
postgres=# COPY customer FROM '/Users/manasamaddi/Desktop/DBMS/DATA_TABLES/customer.csv' DELIMITER '|' CSV;
COPY 1500000
postgres=# COPY orders FROM '/Users/manasamaddi/Desktop/DBMS/DATA_TABLES/orders.csv' DELIMITER '|' CSV;
COPY 15000000
postgres=# COPY lineitem FROM '/Users/manasamaddi/Desktop/DBMS/DATA_TABLES/lineitem.csv' DELIMITER '|' CSV;

```

Figure 2.4: Initial Data Load Execution

```

postgres=# COPY lineitem FROM '/Users/manasamaddi/Desktop/DBMS/DATA_TABLES/lineitem.csv' DELIMITER '|' CSV;
COPY 59986052

```

Figure 2.5: Successful Data Load Execution LINEITEM

Chapter 3

Statistics of the Data

In this section, we will calculate various statistics of the database, including the number of rows in each table, the table sizes in bytes, the number of distinct values for relevant attributes, and the minimum and maximum values for relevant attributes.

3.1 Size of the Database

The total size of the database is: **15GB**. The Figure 3.1 shows the execution of the query that calculates the size of the database.

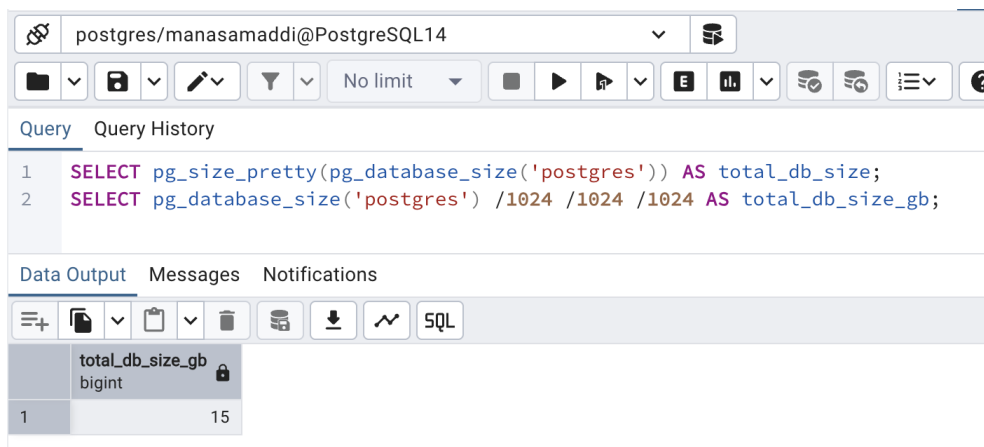


Figure 3.1: Database Size

3.2 Number of Rows in Each Table

The following query, shown in the screenshot (Figure 3.2), calculates the number of rows in each table.

It uses `UNION ALL` to combine the row counts from each table into a single set of results.

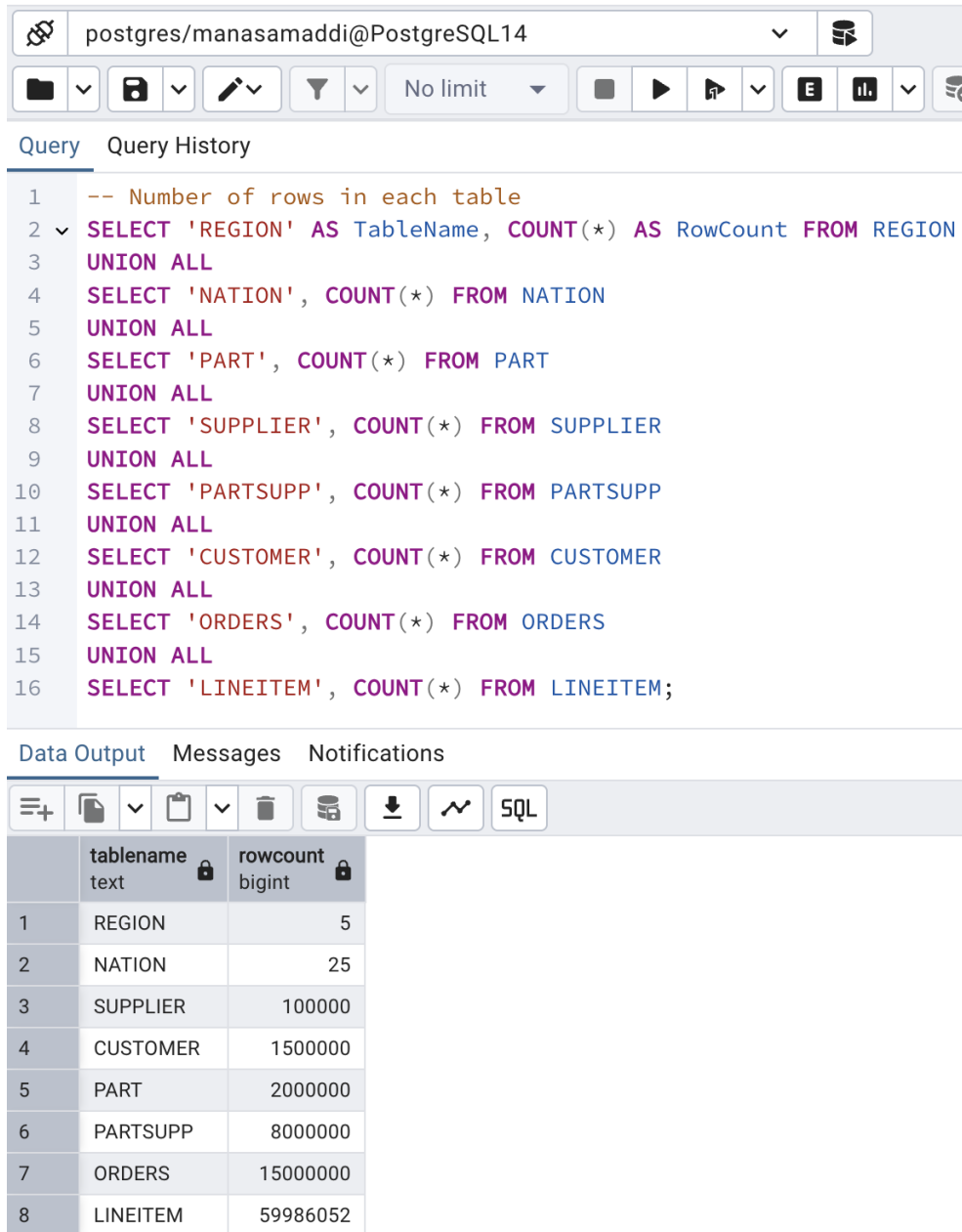


Figure 3.2: Number of rows in each table

3.3 Table Sizes in Bytes

The following query retrieves the size of each table in bytes as shown in Figure 3.3:

```

SELECT
    relname AS TableName,

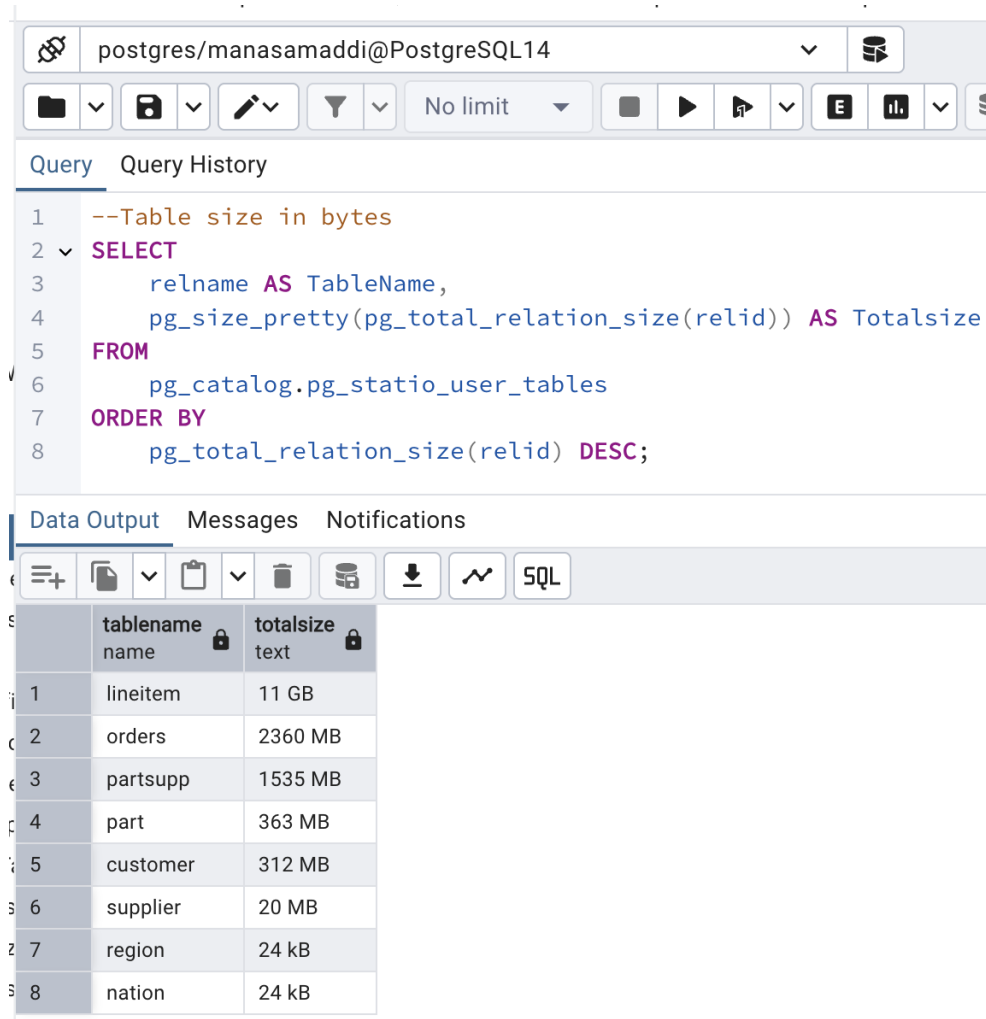
```

```

pg_size_pretty(pg_total_relation_size(relid)) AS
    ↪ TotalSize
FROM
    pg_catalog.pg_statio_user_tables
ORDER BY
    pg_total_relation_size(relid) DESC;

```

This query uses PostgreSQL's `pg_total_relation_size` function to get the total size of each table, and `pg_size_pretty` to format the result into a human-readable string (e.g., in kilobytes or megabytes).



The screenshot shows a PostgreSQL query editor interface. The query is as follows:

```

1  --Table size in bytes
2  SELECT
3      relname AS TableName,
4      pg_size_pretty(pg_total_relation_size(relid)) AS Totalsize
5  FROM
6      pg_catalog.pg_statio_user_tables
7  ORDER BY
8      pg_total_relation_size(relid) DESC;

```

The "Data Output" tab is active, displaying the results of the query in a table with 8 rows. The columns are "tablename" and "totalsize".

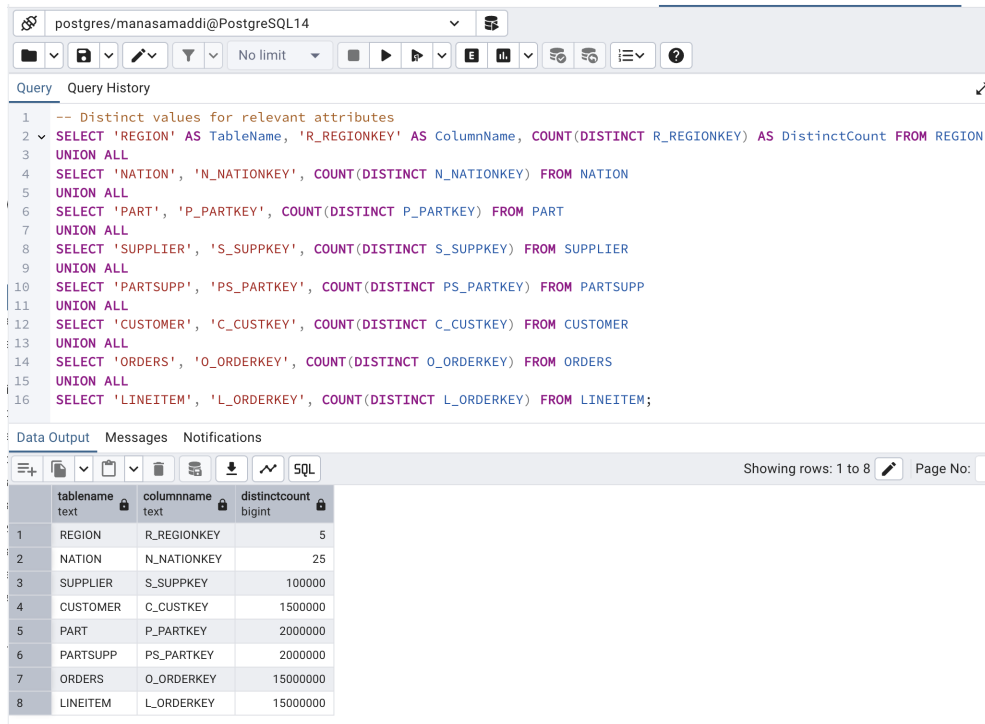
	tablename	totalsize
1	lineitem	11 GB
2	orders	2360 MB
3	partsupp	1535 MB
4	part	363 MB
5	customer	312 MB
6	supplier	20 MB
7	region	24 kB
8	nation	24 kB

Figure 3.3: Table sizes in bytes

3.4 Distinct Values for Relevant Attributes

The following query calculates the number of distinct values for relevant attributes in each table shown in the screenshot below (Figure 3.4).

This query uses the `COUNT(DISTINCT column_name)` function to count the number of unique values in the specified columns.



```
1 -- Distinct values for relevant attributes
2 SELECT 'REGION' AS TableName, 'R_REGIONKEY' AS ColumnName, COUNT(DISTINCT R_REGIONKEY) AS DistinctCount FROM REGION
3 UNION ALL
4 SELECT 'NATION', 'N_NATIONKEY', COUNT(DISTINCT N_NATIONKEY) FROM NATION
5 UNION ALL
6 SELECT 'PART', 'P_PARTKEY', COUNT(DISTINCT P_PARTKEY) FROM PART
7 UNION ALL
8 SELECT 'SUPPLIER', 'S_SUPPKEY', COUNT(DISTINCT S_SUPPKEY) FROM SUPPLIER
9 UNION ALL
10 SELECT 'PARTSUPP', 'PS_PARTKEY', COUNT(DISTINCT PS_PARTKEY) FROM PARTSUPP
11 UNION ALL
12 SELECT 'CUSTOMER', 'C_CUSTKEY', COUNT(DISTINCT C_CUSTKEY) FROM CUSTOMER
13 UNION ALL
14 SELECT 'ORDERS', 'O_ORDERKEY', COUNT(DISTINCT O_ORDERKEY) FROM ORDERS
15 UNION ALL
16 SELECT 'LINEITEM', 'L_ORDERKEY', COUNT(DISTINCT L_ORDERKEY) FROM LINEITEM;
```

	tablename text	columnname text	distinctcount bigint
1	REGION	R_REGIONKEY	5
2	NATION	N_NATIONKEY	25
3	SUPPLIER	S_SUPPKEY	100000
4	CUSTOMER	C_CUSTKEY	1500000
5	PART	P_PARTKEY	2000000
6	PARTSUPP	PS_PARTKEY	2000000
7	ORDERS	O_ORDERKEY	15000000
8	LINEITEM	L_ORDERKEY	15000000

Figure 3.4: Distinct values for relevant attributes

3.5 Minimum and Maximum Values for Relevant Attributes

The following query calculates the minimum and maximum values for relevant attributes in each table as shown in the below screenshot (Figure 3.5).

This query uses the `MIN(column_name)` and `MAX(column_name)` functions to retrieve the minimum and maximum values in the specified columns.

postgres/manasamaddi@PostgreSQL14

Query History

```

1 -- Min And Max values for relevant attributes
2 SELECT 'REGION' AS TableName, 'R_REGIONKEY' AS ColumnName, MIN(R_REGIONKEY) AS Minvalue, MAX(R_REGIONKEY) AS Maxvalue FROM REGION
3 UNION ALL
4 SELECT 'NATION', 'N_NATIONKEY', MIN(N_NATIONKEY), MAX(N_NATIONKEY) FROM NATION
5 UNION ALL
6 SELECT 'PART', 'P_PARTKEY', MIN(P_PARTKEY), MAX(P_PARTKEY) FROM PART
7 UNION ALL
8 SELECT 'SUPPLIER', 'S_SUPPKEY', MIN(S_SUPPKEY), MAX(S_SUPPKEY) FROM SUPPLIER
9 UNION ALL
10 SELECT 'PARTSUPP', 'PS_PARTKEY', MIN(PS_PARTKEY), MAX(PS_PARTKEY) FROM PARTSUPP
11 UNION ALL
12 SELECT 'CUSTOMER', 'C_CUSTKEY', MIN(C_CUSTKEY), MAX(C_CUSTKEY) FROM CUSTOMER
13 UNION ALL
14 SELECT 'ORDERS', 'O_ORDERKEY', MIN(O_ORDERKEY), MAX(O_ORDERKEY) FROM ORDERS
15 UNION ALL
16 SELECT 'LINEITEM', 'L_ORDERKEY', MIN(L_ORDERKEY), MAX(L_ORDERKEY) FROM LINEITEM;
17

```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1 of 1

	tablename text	columnname text	minvalue integer	maxvalue integer
1	NATION	N_NATIONKEY	0	24
2	REGION	R_REGIONKEY	0	4
3	PART	P_PARTKEY	1	2000000
4	SUPPLIER	S_SUPPKEY	1	100000
5	PARTSUPP	PS_PARTKEY	1	2000000
6	CUSTOMER	C_CUSTKEY	1	1500000
7	ORDERS	O_ORDERKEY	1	60000000
8	LINEITEM	L_ORDERKEY	1	60000000

Figure 3.5: Minimum and maximum values for relevant attributes

Chapter 4

Query Schema and Analysis Before Optimization

4.1 Pricing Summary Report Query (Q1)

Objective: This query reports aggregated pricing and discount statistics for lineitem shipped within a specified time range, grouped by return flag and line status. It includes sums, averages, and counts.

Involved Tables: LINEITEM

Join Conditions: None

Filters:

- `L_SHIPDATE <= DATE '1998-12-01' - INTERVAL '90' DAY`

Group By: L_RETURNFLAG, L_LINESTATUS

Observations:

- Heavy scan on LINEITEM table.
- Aggregations can be optimized using indexes on L_SHIPDATE.

The screenshot shows a PostgreSQL query editor with a query titled "Pricing summary Report Query(Q1) before Optimization." The query is a SELECT statement that calculates various metrics for line items, including return flags, line statuses, quantities, base prices, discounted prices, charges, average quantities, average prices, average discounts, and order counts. The results are grouped by return flag and line status, and ordered by return flag and line status.

The query is as follows:

```

1  -- Pricing summary Report Query(Q1) before Optimization.
2  SELECT
3      L_RETURNFLAG AS RETURN_FLAG,
4      L_LINESTATUS AS LINE_STATUS,
5      SUM(L_QUANTITY) AS TOTAL_QUANTITY,
6      SUM(L_EXTENDEDPRICE) AS TOTAL_BASE_PRICE,
7      SUM(L_EXTENDEDPRICE * (1 - L_DISCOUNT)) AS TOTAL_DISCOUNTED_PRICE,
8      SUM(L_EXTENDEDPRICE * (1 - L_DISCOUNT) * (1 + L_TAX)) AS TOTAL_CHARGE,
9      AVG(L_QUANTITY) AS AVERAGE_QUANTITY,
10     AVG(L_EXTENDEDPRICE) AS AVERAGE_PRICE,
11     AVG(L_DISCOUNT) AS AVERAGE_DISCOUNT,
12     COUNT(*) AS ORDER_COUNT
13 FROM
14     LINEITEM
15 WHERE
16     L_SHIPDATE <= DATE '1998-12-01' - INTERVAL '90' DAY
17 GROUP BY
18     L_RETURNFLAG,
19     L_LINESTATUS
20 ORDER BY
21     L_RETURNFLAG,
22     L_LINESTATUS;

```

The results are displayed in a table with the following columns:

	return_flag character (1)	line_status character (1)	total_quantity numeric	total_base_price numeric	total_discounted_price numeric	total_charge numeric	average_quantity numeric	average_price numeric	average_discount numeric	order_count bigint
1	A	F	377518399	566065727797.25	537759104278.0656	559276670892.116819	25.5009751030070973	38237.151008958546	0.05000657454024320463	14804077
2	N	F	9851614	14767438399.17	14028805792.2114	14590490998.366737	25.5224483028409474	38257.810660081140	0.04997336773765667180	385998
3	N	O	743124873	1114302286901.88	1058580922144.9638	1100937000170.591854	25.4980758706893147	38233.902923481810	0.05000081182113130603	29144351
4	R	F	377732830	566431054976.00	538110922664.7677	559634780885.086257	25.5083847896801383	38251.219273559761	0.04999679231408742045	14808183

The table shows 4 rows of data. The total rows are 4. The query is complete in 00:00:25.869 seconds. The page number is 1 of 1. The table is displayed in LF mode, column 1.

Figure 4.1: Pricing Summary Report (Q1)

4.2 Shipping Priority Query (Q3)

Objective: This query retrieves the top 10 orders with the highest revenue, filtered by a specific shipping date, and grouped by order and customer.

Involved Tables: CUSTOMER, ORDERS, LINEITEM

Join Conditions:

- C_CUSTKEY = O_CUSTKEY
- O_ORDERKEY = L_ORDERKEY

Filters:

- C_MKTSEGMENT = 'BUILDING'
- O_ORDERDATE < DATE '1995-03-15'
- L_SHIPDATE > DATE '1995-03-15'

Group By: O_ORDERKEY, O_ORDERDATE, O_SHIPPRIORITY

Observations:

- Significant join and filtering operations.
- Beneficial to index O_ORDERDATE, L_SHIPDATE, and C_MKTSEGMENT.

Query

Query History

1

-- Shipping Priority Query (Q3) before Optimization

2

SELECT

3

L_ORDERKEY AS Order_Key,

4

SUM (L_EXTENDEDPRICE * (1-L_DISCOUNT)) AS revenue,

5

O_ORDERDATE AS Order_date,

6

O_SHIPPRIORITY AS ship_priority

7

FROM

8

CUSTOMER,

9

ORDERS,

10

LINEITEM

11

WHERE

12

C_MKTSEGMENT = 'BUILDING'

13

AND C_CUSTKEY = O_CUSTKEY

14

AND L_ORDERKEY = O_ORDERKEY

15

AND O_ORDERDATE < DATE '1995-03-15'

16

AND L_SHIPDATE > DATE '1995-03-15'

17

GROUP BY

18

L_ORDERKEY,

19

O_ORDERDATE,

20

O_SHIPPRIORITY

21

ORDER BY

22

revenue DESC,

23

O_ORDERDATE

24

LIMIT 10;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 10

Page No: 1 of 1

	order_key integer	revenue numeric	order_date date	ship_priority integer
1	4791171	440715.2185	1995-02-23	0
2	46678469	439855.3250	1995-01-27	0
3	23906758	432728.5737	1995-03-14	0
4	23861382	428739.1368	1995-03-09	0
5	59393639	426036.0662	1995-02-12	0
6	3355202	425100.6657	1995-03-04	0
7	9806272	425088.0568	1995-03-13	0
8	22810436	423231.9690	1995-01-02	0
Total rows: 10 Query complete 00:00:25.309				

LF Ln 2, Col 5

Figure 4.2: Shipping Priority (Q3)

4.3 Local Supplier Volume Query (Q5)

Objective: Computes revenue generated by local suppliers (the same nation) in a specific region within a date range.

Involved Tables: CUSTOMER, ORDERS, LINEITEM, SUPPLIER, NATION, REGION

Join Conditions:

- C_CUSTKEY = O_CUSTKEY
- O_ORDERKEY = L_ORDERKEYy
- L_SUPPKEY = S_SUPPKEY
- C_NATIONKEY = S_NATIONKEY
- N_NATIONKEY = S_NATIONKEYy
- N_REGIONKEY = R_REGIONKEY

Filters:

- `R_NAME = 'ASIA'`
- `O_ORDERDATE BETWEEN '1994-01-01' AND '1995-01-01'`

Group By: `N_NAME`

Observations:

- Multi-way joins and heavy filtering.
- Join reordering and predicate pushdown are key optimization opportunities.

- C_CUSTKEY = O_CUSTKEY
- O_ORDERKEY = L_ORDERKEY
- L_SUPPKEY = S_SUPPKEY
- C_NATIONKEY = N1.N_NATIONKEY
- S_NATIONKEY = N2.N_NATIONKEY

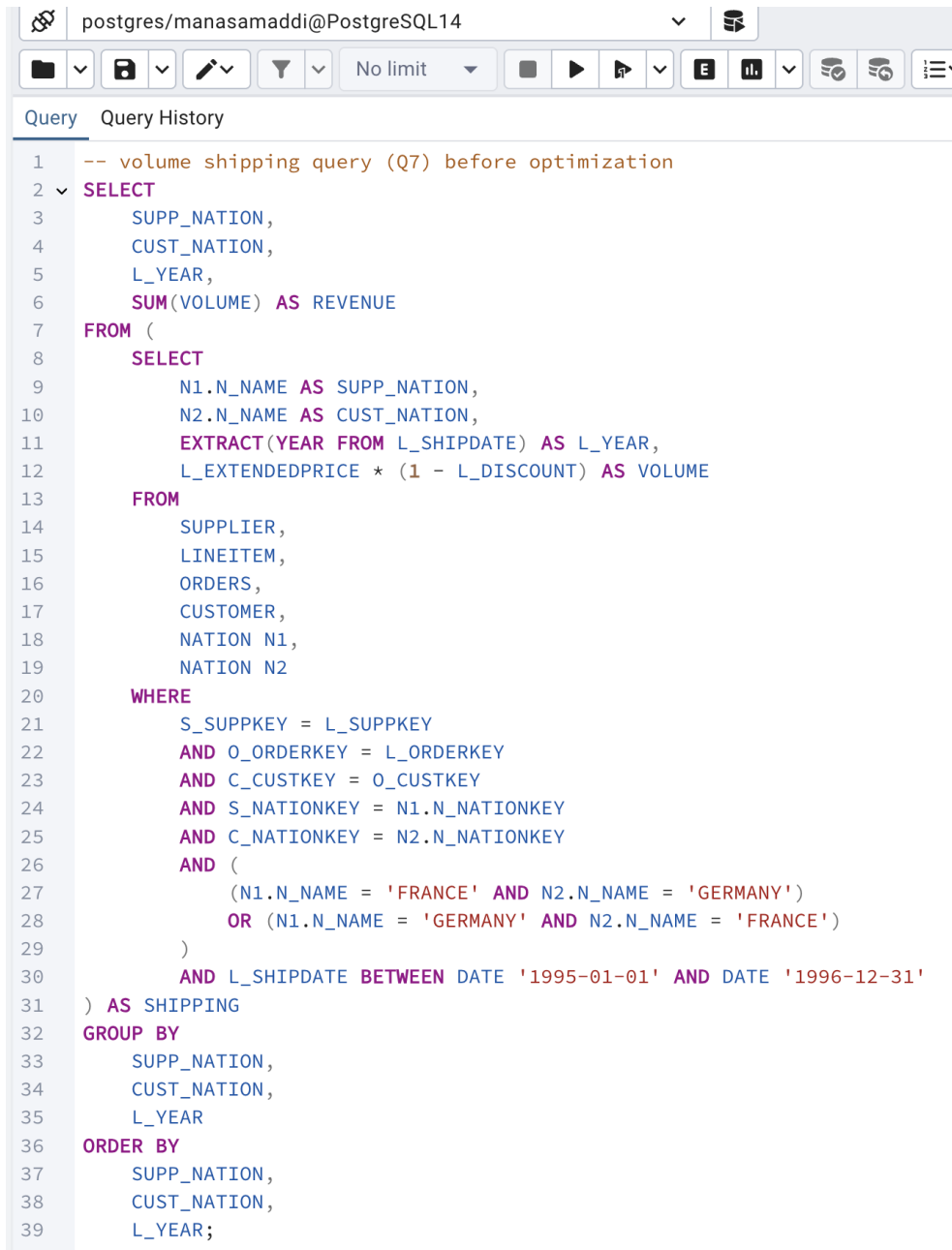
Filters:

- N1.N_NAME = 'FRANCE' AND N2.N_NAME = 'GERMANY' or vice versa
- L_SHIPDATE BETWEEN '1995-01-01' AND '1996-12-31'

Group By: SUPP_NATION, CUST_NATION, L_SHIPYEAR

Observations:

- Requires date extraction and multiple nation joins.
- Use of derived tables or materialized views could improve performance.



```

1 -- volume shipping query (Q7) before optimization
2 SELECT
3     SUPP_NATION,
4     CUST_NATION,
5     L_YEAR,
6     SUM(VOLUME) AS REVENUE
7 FROM (
8     SELECT
9         N1.N_NAME AS SUPP_NATION,
10        N2.N_NAME AS CUST_NATION,
11        EXTRACT(YEAR FROM L_SHIPDATE) AS L_YEAR,
12        L_EXTENDEDPRICE * (1 - L_DISCOUNT) AS VOLUME
13 FROM
14     SUPPLIER,
15     LINEITEM,
16     ORDERS,
17     CUSTOMER,
18     NATION N1,
19     NATION N2
20 WHERE
21     S_SUPPKEY = L_SUPPKEY
22     AND O_ORDERKEY = L_ORDERKEY
23     AND C_CUSTKEY = O_CUSTKEY
24     AND S_NATIONKEY = N1.N_NATIONKEY
25     AND C_NATIONKEY = N2.N_NATIONKEY
26     AND (
27         (N1.N_NAME = 'FRANCE' AND N2.N_NAME = 'GERMANY')
28         OR (N1.N_NAME = 'GERMANY' AND N2.N_NAME = 'FRANCE')
29     )
30     AND L_SHIPDATE BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
31 ) AS SHIPPING
32 GROUP BY
33     SUPP_NATION,
34     CUST_NATION,
35     L_YEAR
36 ORDER BY
37     SUPP_NATION,
38     CUST_NATION,
39     L_YEAR;

```

Figure 4.4: Volume shipping (Q7)

4.5 Top Supplier Query (Q15)

Objective: Identifies the supplier with the highest total revenue over a time window.

Involved Tables: SUPPLIER, LINEITEM

Join Conditions:

- S_SUPPKEY = L_SUPPKEY

Filters:

- L_SHIPDATE BETWEEN '1996-01-01' AND '1996-04-01'

Group By: S_SUPPKEY, S_NAME, S_ADDRESS, S_PHONE

Observations:

- Straightforward aggregation and ranking.
- Benefit from indexing L_SHIPDATE.

Query

Query History

```

1  --Top supplier query (Q15)
2
3  -- STEP 1: CREATE A VIEW THAT CALCULATES TOTAL REVENUE PER SUPPLIER IN THE GIVEN QUART
4
5  CREATE OR REPLACE VIEW REVENUE_Q15 AS
6  SELECT
7      L_SUPPKEY AS SUPPLIER_NO,
8      SUM(L_EXTENDEDPRICE * (1 - L_DISCOUNT)) AS TOTAL_REVENUE
9  FROM
10     LINEITEM
11  WHERE
12     L_SHIPDATE >= DATE '1996-01-01'
13     AND L_SHIPDATE < DATE '1996-01-01' + INTERVAL '3 MONTHS'
14  GROUP BY
15     L_SUPPKEY;
16
17  -- STEP 2: SELECT SUPPLIER DETAILS FOR THE SUPPLIER(S) WITH THE MAXIMUM REVENUE IN THE QUART
18  SELECT
19     s.s_suppkey,
20     s.s_name,
21     s.s_address,
22     s.s_phone,
23     r.total_revenue
24  FROM
25     supplier s
26  JOIN
27     revenue_q15 r ON s.s_suppkey = r.supplier_no
28  WHERE
29     r.total_revenue = (
30         SELECT MAX(total_revenue) FROM revenue_q15
31     )
32  ORDER BY
33     s.s_suppkey;

```

Data Output

Messages

Notifications

	s_suppkey integer	s_name character (25)	s_address character varying (40)	s_phone character (15)	total_revenue numeric
1	69998	Supplier#0000699...	,ZT4VX2ygg9dLsG298SbYYSVUqeHjhLSRVxNxGv	16-386-278-9829	2194132.8166

Figure 4.5: Top supplier query (Q15)

4.6 Global Sales Opportunity Query (Q22)

Objective: Finds potential customer segments with small orders and evaluates their contribution to revenue.

Involved Tables: CUSTOMER, ORDERS

Join Conditions:

- C_CUSTKEY = O_CUSTKEY

Filters:

- C_ACCTBAL > 0
- Subquery: customers with fewer than average orders

Group By: COUNTRYCODE

Observations:

- Involves correlated subqueries and grouping.
- Potential performance improvements via materialized views or subquery flattening.

QueryQuery History

1-- Global Sales Opportunity Query (Q22)

2SELECT

3COUNTRYCODE,

4COUNT(*) AS NUMBER_CUSTOMER,

5SUM(C_ACCTBAL) AS TOTAL_ACCOUNT_BALANCE

6FROM (

7SELECT

8SUBSTRING(C_PHONE FROM 1 FOR 2) AS COUNTRYCODE,

9C_ACCTBAL

10FROM

11CUSTOMER

12WHERE

13SUBSTRING(C_PHONE FROM 1 FOR 2) IN ('13', '31', '23', '29', '30', '18', '17')

14AND C_ACCTBAL > (

15SELECT

16AVG(C_ACCTBAL)

17FROM

18CUSTOMER

19WHERE

20C_ACCTBAL > 0.00

21AND SUBSTRING(C_PHONE FROM 1 FOR 2) IN ('13', '31', '23', '29', '30', '18', '17')

22)

23AND NOT EXISTS (

24SELECT

25*

26FROM

27ORDERS

28WHERE

29O_CUSTKEY = C_CUSTKEY

30)

31) AS CUSTSALE

32GROUP BY

33COUNTRYCODE

34ORDER BY

35COUNTRYCODE;

Data OutputMessagesNotifications

+

SQL

<

Figure 4.6: Global sales opportunity (Q22)

Before applying any optimizations, I executed all six selected queries on the base TPC-H dataset without any indexes or materialized views. The table 4.1 lists each query along with a short description and the time it took to run in this unoptimized state. These values serve as a baseline to compare with the performance after optimization.

Table 4.1: Execution Time for Queries Before Optimization

Query ID	Query Description	Execution Time (s)
Q1	Pricing Summary Report	29.058
Q3	Shipping Priority	21.897
Q5	Local Supplier Volume	5.749
Q7	Volume Shipping by Nation	3.834
Q15	Top Supplier	5.474
Q22	Global Sales Opportunity	2.693413
Total	—	68.708

Chapter 5

Query Optimization

In this chapter, we describe how we optimized the set of TPC-H benchmark queries selected for our project: Q1, Q3, Q5, Q7, Q15, and Q22. The objective was not to optimize each query in isolation, but to identify shared structures and strategies that reduce the total execution time and resource usage for the full set. This aligns with the optimization constraints described in the project specification (DW-CS-24-25), where total performance across the set is the key metric.

5.1 Optimization Strategy Overview

To improve the overall performance of our selected TPC-H queries, we adopted a systematic approach to optimization. The process was structured as follows:

1. We began by evaluating the baseline performance of all six queries without any indexes or materialized views. This step was completed in the previous chapter and served as a reference point for measuring improvements.
2. Next, we identified columns that were frequently used in filtering or join conditions across multiple queries. We created indexes on these columns to speed up access and reduce full-table scans.
3. For queries with complex joins and reusable aggregations, we designed materialized views to precompute and store intermediate results. These views were intentionally built to support more than one query, maximizing their benefit.
4. Finally, we measured the performance impact of our optimizations by recording the query execution time, index/view creation time, and the disk space used. This helped us assess the trade-off between query speed and storage cost, and to confirm that our strategy improved the performance of the entire query set, not just individual queries.

5.2 Optimization: Discussion and Strategy

In this section, we present a detailed discussion of how we approached the optimization of the selected TPC-H queries (Q1, Q3, Q5, Q7, Q15, Q22). Instead of treating each query as an isolated problem, we collaborated to identify common patterns, shared columns, and frequent filters that could be leveraged across the set.

Our optimization strategy unfolded through thoughtful discussion. We frequently proposed and debated different approaches, reasoning through their benefits and trade-offs in terms of performance, build time, and storage space. A recurring principle we agreed on was to favor solutions that improve multiple queries, not just one.

We knew from the beginning that our main goal was to reduce overall execution time for the entire query set — not just for individual queries. That meant avoiding one-off tweaks and instead looking for opportunities shared across multiple queries, like recurring WHERE conditions, commonly joined tables, and large datasets that could benefit from indexing or pre-aggregation.

We also agreed that we couldn't just index everything. Index creation time and storage space were concerns, and PostgreSQL already indexes primary keys automatically. So we focused on columns that are repeatedly filtered or joined but are not primary keys — especially in large tables like `LINEITEM` and `ORDERS`.

Join Strategy: Why We Used Plain JOINS

In all optimized queries, we used the implicit or classical form of joins (i.e., `FROM A, B WHERE A.key = B.key`), which is functionally equivalent to `INNER JOIN`. For example, in Query 3 (Shipping Priority), the join is written as:

```
FROM CUSTOMER, ORDERS, LINEITEM
WHERE C_CUSTKEY = O_CUSTKEY
AND L_ORDERKEY = O_ORDERKEY
```

This form avoids a Cartesian product because the join conditions are properly specified in the WHERE clause. Although this syntax can produce all combinations of rows (a Cartesian product) if no WHERE conditions are present, the TPC-H queries ensure that all joins are intentional and restrictive.

We didn't explicitly use `INNER JOIN` or `OUTER JOIN` because:

All queries in TPC-H are designed to work with matched data only, meaning rows must exist in all joined tables — which makes implicit joins or `INNER JOINS`

ideal.

Using `OUTER JOIN` would introduce Nulls and unmatched rows, which are not required for any of these decision-support-style queries.

Using classical syntax made the queries more consistent with the original TPC-H query formulations.

PostgreSQL's query planner is smart enough to optimize both styles similarly. Therefore, there was no performance disadvantage in using the implicit join syntax, and sticking with it helped maintain the benchmark's intended structure.

Key Observations:

Most queries filter by `L_SHIPDATE`, `O_ORDERDATE`, or `C_MKTSEGMENT` — strong candidates for indexing.

Several queries (like Q3, Q5, and Q7) rely on joining large tables (`LINEITEM`, `ORDERS`) using foreign key columns — indexing those keys improves join performance.

Some queries (like Q7 and Q15) involve expensive aggregations and joins — perfect use cases for materialized views.

Why Not Index Primary Keys?

Since PostgreSQL already creates B-tree indexes on primary keys like `C_CUSTKEY` or `O_ORDERKEY`, we didn't spend time recreating them. We focused on optimizing foreign keys and high-selectivity fields that PostgreSQL doesn't index automatically but are heavily used in joins and filters.

For example, we created an index on `LINEITEM(L_SHIPDATE)` because it appears in the `WHERE` clause and helps filter billions of rows. Since Q1, Q3, Q5, Q7, and Q15 all use this filter, this single index supports multiple queries efficiently.

Similarly, we indexed `CUSTOMER(C_MKTSEGMENT)` and `orders(O_ORDERDATE)` because they are used for filtering. When combined with `L_SHIPDATE` from Q1, this gives Q3 a highly optimized path across all its joins.

Queries like Q7 and Q15 involve complex joins and aggregations. So, we discussed pre-aggregating their heavy parts using materialized views. These views not only sped up Q7 and Q15 but also provided partial support for Q5. This reuse of precomputed logic fits the spirit of system-level optimization.

Q22 has more specialized filtering logic. It filters on expressions like `SUBSTRING(C_PHONE,`

1, 2) and compares `C_ACCTBAL` against an average, while ensuring the customer never placed an order. We indexed the relevant expression and foreign keys to accelerate this filtering.

Overall, we consistently asked ourselves: “Does this help more than one query?” We only created indexes and materialized views that passed this test. This aligns with the DW-CS-24-25 project rule of optimizing the set, not individual queries in isolation.

5.2.1 Query 1: Pricing Summary Report

Optimization: Index on `lineitem(L_SHIPDATE)`.

Reason for Optimization: Query 1 heavily filters records based on the shipping date. This same condition is also used in Q3, Q5, Q7, and Q15. We decided to create an index on `L_SHIPDATE` to accelerate filtering in all those queries. Since `LINEITEM` is the largest table in the schema, indexing its date column reduces the number of rows scanned and improves overall join performance.

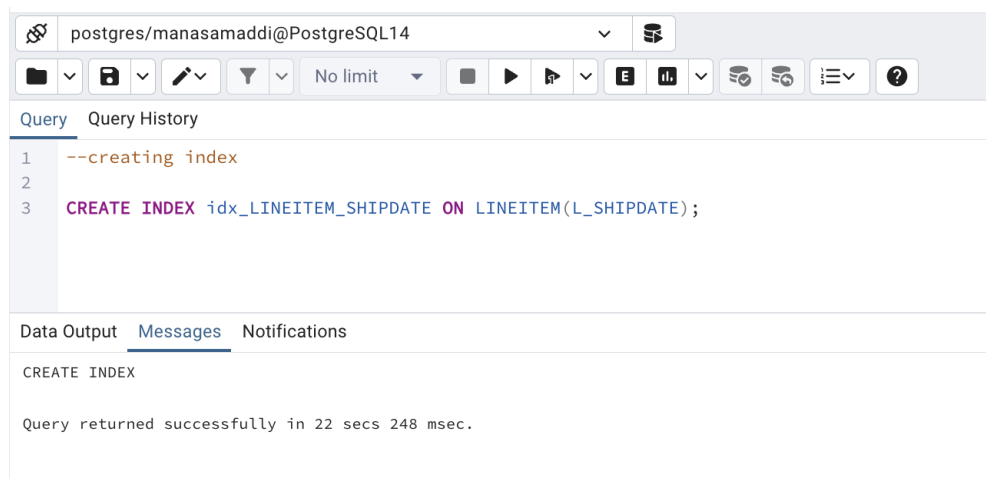


Figure 5.1: Creating Index for Q1

Query

Query History

1

-- Pricing Summary Report (Q1) optimization

2

EXPLAIN ANALYZE

3

SELECT

4

L_RETURNFLAG AS RETURN_FLAG,

5

L_LINESTATUS AS LINE_STATUS,

6

SUM(L_QUANTITY) AS TOTAL_QUANTITY,

7

SUM(L_EXTENDEDPRI) AS TOTAL_BASE_PRICE,

8

SUM(L_EXTENDEDPRI *(1 - L_DISCOUNT)) AS TOTAL_DISCOUNTED_PRICE,

9

SUM(L_EXTENDEDPRI *(1 - L_DISCOUNT)*(1+ L_TAX)) AS TOTAL_CHARGE,

10

AVG(L_QUANTITY) AS AVERAGE_QUANTITY,

11

AVG(L_EXTENDEDPRI) AS AVERAGE_PRICE,

12

AVG(L_DISCOUNT) AS AVERAGE_DISCOUNT,

13

COUNT(*) AS ORDER_COUNT

14

FROM

15

LINEITEM

16

WHERE

17

L_SHIPDATE <= DATE '1998-12-01' - INTERVAL '90' DAY

18

GROUP BY

19

L_RETURNFLAG,

20

L_LINESTATUS

21

ORDER BY

22

L_RETURNFLAG,

23

L_LINESTATUS;

Data Output

Messages

Notifications

≡

+

📄

▼

📄

▼

🗑

📦

⬇

📶

SQL

QUERY PLAN

text

1

Finalize GroupAggregate (cost=2300231.87..2300233.82 rows=6 width=236) (actual time=21277.038..21277.850 rows=4)

2

Group Key: l_returnflag, l_linestatus

Figure 5.2: Optimized Query 1 Execution

5.2.2 Query 3: Shipping Priority

Optimization: Indexes on CUSTOMER(C_MKTSEGMENT), ORDERS(O_ORDERDATE), and LINEITEM(L_SHIPDATE).

Reason for Optimization: Q3 filters customers by market segment and order date, and retrieves recently shipped items. These filters can benefit from indexes to avoid full-table scans on CUSTOMER, ORDERS, and LINEITEM. Additionally, L_SHIPDATE is reused from the optimization in Q1. Indexing these columns helps PostgreSQL execute the joins more efficiently and leverages index scans instead of sequential scans.

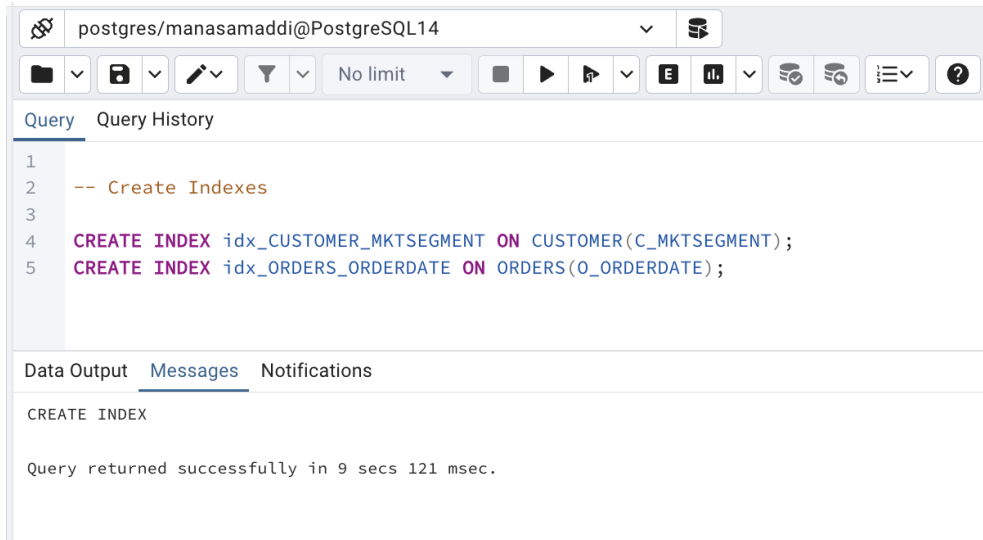


Figure 5.3: Creating Index for Q3

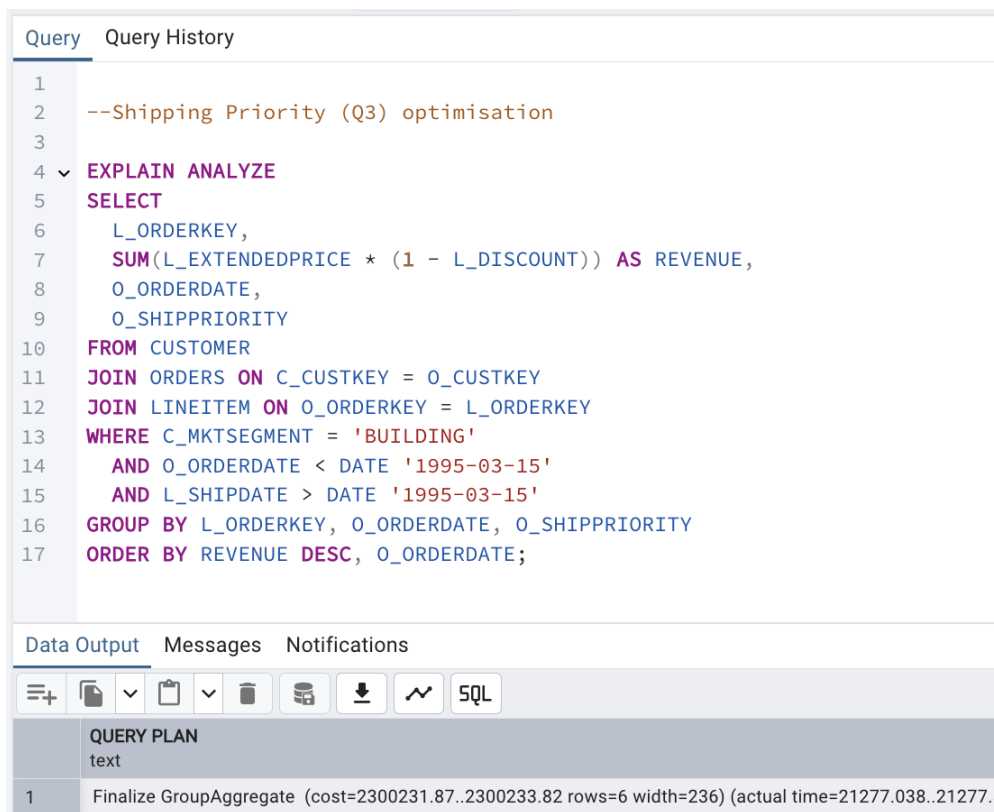
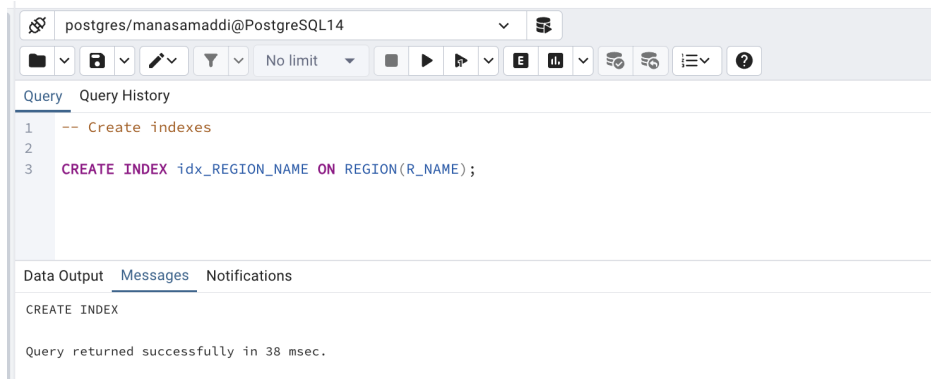


Figure 5.4: Optimized Query 3 Execution

5.2.3 Query 5: Local Supplier Volume

Optimization: Indexes on REGION(R_NAME), ORDERS(O_ORDERDATE), and LINEITEM(L_SHIPDATE).

Reason for Optimization: The query filters on orders placed in a specific year and customers located in a given region. Indexes on `R_NAME` and `O_ORDERDATE` speed up filtering on both dimension and fact tables. Since `LINEITEM` is already indexed by `L_SHIPDATE`, we reused that to support the join conditions.



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the user is 'postgres/manasamaddi@PostgreSQL14'. Below the toolbar, the 'Query' tab is active, showing a SQL script with two lines: a comment '-- Create indexes' and a command 'CREATE INDEX idx_REGION_NAME ON REGION(R_NAME);'. The 'Data Output' tab is also visible, showing the command 'CREATE INDEX' and a message 'Query returned successfully in 38 msec.'

```

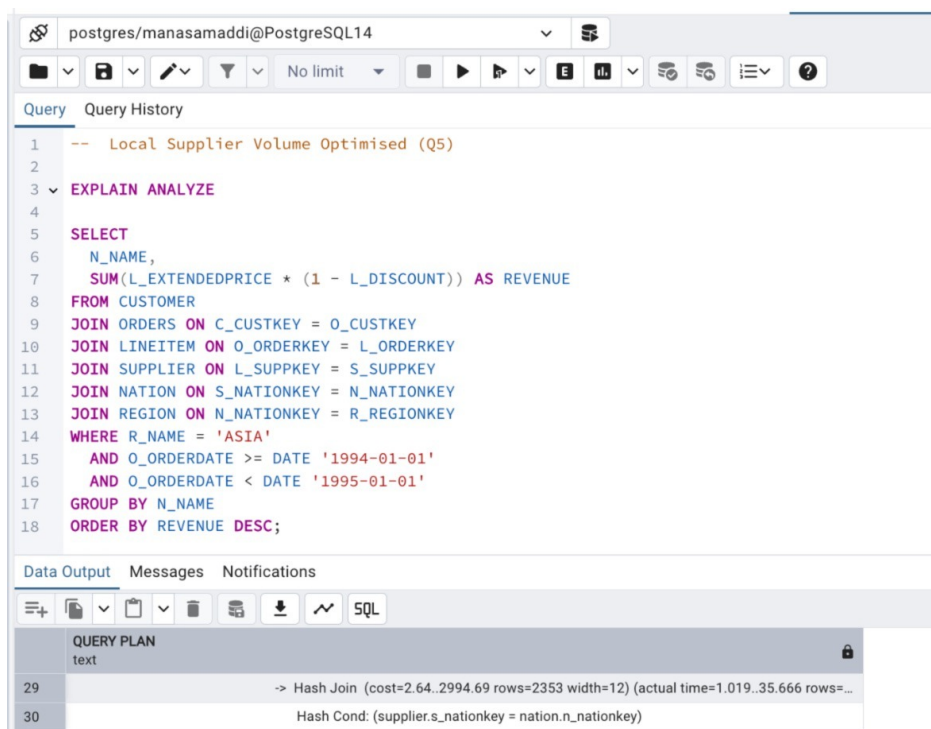
1  -- Create indexes
2
3  CREATE INDEX idx_REGION_NAME ON REGION(R_NAME);

```

CREATE INDEX

Query returned successfully in 38 msec.

Figure 5.5: Creating Index for Q5



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the user is 'postgres/manasamaddi@PostgreSQL14'. Below the toolbar, the 'Query' tab is active, showing a SQL script for 'Local Supplier Volume Optimised (Q5)'. The script includes an 'EXPLAIN ANALYZE' command followed by a complex query with multiple joins and filters. The 'Data Output' tab is also visible, showing the 'QUERY PLAN' for the query, which includes a 'Hash Join' operation with its cost, rows, and width.

```

1  -- Local Supplier Volume Optimised (Q5)
2
3  EXPLAIN ANALYZE
4
5  SELECT
6      N_NAME,
7      SUM(L_EXTENDEDPRICE * (1 - L_DISCOUNT)) AS REVENUE
8  FROM CUSTOMER
9  JOIN ORDERS ON C_CUSTKEY = O_CUSTKEY
10 JOIN LINEITEM ON O_ORDERKEY = L_ORDERKEY
11 JOIN SUPPLIER ON L_SUPPKEY = S_SUPPKEY
12 JOIN NATION ON S_NATIONKEY = N_NATIONKEY
13 JOIN REGION ON N_NATIONKEY = R_REGIONKEY
14 WHERE R_NAME = 'ASIA'
15        AND O_ORDERDATE >= DATE '1994-01-01'
16        AND O_ORDERDATE < DATE '1995-01-01'
17 GROUP BY N_NAME
18 ORDER BY REVENUE DESC;

```

QUERY PLAN

text

29 --> Hash Join (cost=2.64..2994.69 rows=2353 width=12) (actual time=1.019..35.666 rows=...

30 Hash Cond: (supplier.s_nationkey = nation.n_nationkey)

Figure 5.6: Optimized Query 5 Execution

5.2.4 Query 7: Volume Shipping by Nation

Optimization: Materialized view of `mv_export_import`, plus shared index on `LINEITEM(L_SHIPDATE)`.

Reason for Optimization: This query involves multiple joins across CUSTOMER, ORDERS, SUPPLIER, LINEITEM, NATION, and REGION. The joins are complex and expensive. Instead of executing them repeatedly, we created a materialized view that pre-joins and filters the data needed by Q7. This materialized view can also benefit Q5 or similar queries with nation-level aggregations.

The screenshot shows a PostgreSQL IDE window titled 'postgres/manasamaddi@PostgreSQL14'. The 'Query' tab is active, displaying the following SQL code:

```

1
2  -- Create Materialized View for Q7
3
4  CREATE MATERIALIZED VIEW mv_export_import AS
5  SELECT
6      L_EXTENDEDPRICE * (1 - L_DISCOUNT) AS REVENUE,
7      N1.N_NAME AS SUPP_NATION,
8      N2.N_NAME AS CUST_NATION,
9      L_SHIPDATE
10 FROM SUPPLIER S
11 JOIN NATION N1 ON S.S_NATIONKEY = N1.N_NATIONKEY
12 JOIN LINEITEM L ON S.S_SUPPKEY = L.L_SUPPKEY
13 JOIN ORDERS O ON L.L_ORDERKEY = O.O_ORDERKEY
14 JOIN CUSTOMER C ON O.O_CUSTKEY = C.C_CUSTKEY
15 JOIN NATION N2 ON C.C_NATIONKEY = N2.N_NATIONKEY;
16

```

Below the query, the 'Data Output' tab shows the result of the query: 'SELECT 59986052'. A message at the bottom states: 'Query returned successfully in 2 min 31 secs.'

Figure 5.7: Creating Materialized view for Q7

The screenshot shows a PostgreSQL IDE window titled 'postgres/manasamaddi@PostgreSQL14'. The 'Query' tab is active, displaying the following SQL code:

```

1  -- volume shipping query Optimised (Q7)
2
3
4  EXPLAIN ANALYZE
5  SELECT
6      SUPP_NATION,
7      CUST_NATION,
8      EXTRACT(YEAR FROM L_SHIPDATE) AS L_YEAR,
9      SUM(REVENUE) AS SUM_REVENUE
10 FROM mv_export_import
11 WHERE ((SUPP_NATION = 'FRANCE' AND CUST_NATION = 'GERMANY')
12        OR (SUPP_NATION = 'GERMANY' AND CUST_NATION = 'FRANCE'))
13        AND L_SHIPDATE BETWEEN DATE '1995-01-01' AND DATE '1996-12-31'
14 GROUP BY SUPP_NATION, CUST_NATION, L_YEAR
15 ORDER BY SUPP_NATION, CUST_NATION, L_YEAR;

```

Below the query, the 'Data Output' tab shows the execution plan. The plan is as follows:

```

QUERY PLAN
text
1  Finalize GroupAggregate (cost=1076164.34..1083900.96 rows=57220 width=80) (actual time=1609.771..1612.161 rows=4 loops=1)
2    Group Key: supp_nation, cust_nation, (EXTRACT(year FROM L_shipdate))
3    -> Gather Merge (cost=1076164.34..1082435.78 rows=48550 width=80) (actual time=1609.084..1612.149 rows=12 loops=1)
4      Workers Planned: 2
5      Workers Launched: 2
6      -> Partial GroupAggregate (cost=1075164.32..1075831.88 rows=24275 width=80) (actual time=1602.913..1605.091 rows=4 loops=3)
7        Group Key: supp_nation, cust_nation, (EXTRACT(year FROM L_shipdate))
8        -> Sort (cost=1075164.32..1075225.01 rows=24275 width=56) (actual time=1602.101..1602.536 rows=19455 loops=3)
9          Sort Key: supp_nation, cust_nation, (EXTRACT(year FROM L_shipdate))
10         Sort Method: quicksort Memory: 2306kB
11         Worker 0: Sort Method: quicksort Memory: 2273kB
12         Worker 1: Sort Method: quicksort Memory: 2286kB
13         -> Parallel Seq Scan on mv_export_import (cost=0.00..1073396.23 rows=24275 width=56) (actual time=0.370..1595.656 rows=19455 loops=3)
14           Filter: (((L_shipdate >= '1995-01-01'::date) AND (L_shipdate <= '1996-12-31'::date)) AND (((supp_nation)::text = 'FRANCE'::text) AND ((cust_nation)::text = 'GERMANY'::text) OR (((supp_nation)::text = 'GERMANY'::text) AND ((cust_nation)::text = 'FRANCE'::text)))
15           Rows Removed by Filter: 19975896
16         Planning Time: 1.302 ms
17         Execution Time: 1612.331 ms

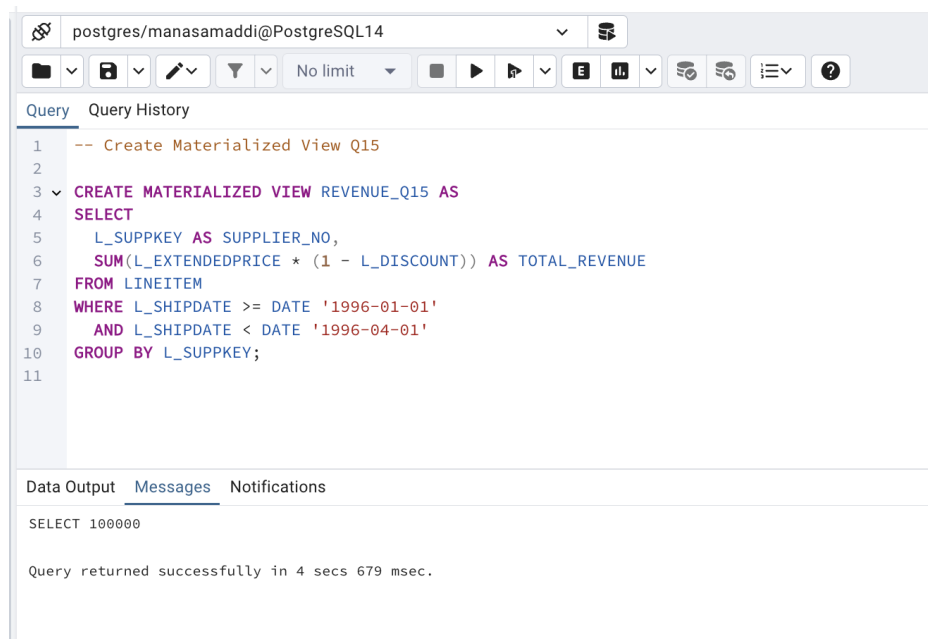
```

Figure 5.8: Optimized Query 7 Execution

5.2.5 Query 15: Top Supplier

Optimization: Materialized view REVENUE_Q15 pre-aggregating revenue grouped by supplier.

Reason for Optimization: The core of Q15 is to rank suppliers by revenue within a time frame. This calculation is reused often in analytical queries. Creating a pre-aggregated materialized view drastically reduces computation time and can also benefit similar queries involving supplier-level revenue breakdowns.



The screenshot shows a PostgreSQL query editor interface. At the top, the connection is identified as 'postgres/manasamaddi@PostgreSQL14'. Below the connection bar is a toolbar with icons for file operations, query execution, and other database functions. The main area is divided into two tabs: 'Query' and 'Query History'. The 'Query' tab is active, displaying a SQL script to create a materialized view. The script is as follows:

```
1  -- Create Materialized View Q15
2
3  CREATE MATERIALIZED VIEW REVENUE_Q15 AS
4  SELECT
5      L_SUPPKEY AS SUPPLIER_NO,
6      SUM(L_EXTENDEDPRICE * (1 - L_DISCOUNT)) AS TOTAL_REVENUE
7  FROM LINEITEM
8  WHERE L_SHIPDATE >= DATE '1996-01-01'
9        AND L_SHIPDATE < DATE '1996-04-01'
10 GROUP BY L_SUPPKEY;
11
```

Below the query editor, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the result of the query execution:

```
SELECT 100000
```

Below the messages, a status message indicates: 'Query returned successfully in 4 secs 679 msec.'

Figure 5.9: Creating Materialized view for Q15

postgres/manasamaddi@PostgreSQL14	
No limit	
Query Query History	
1	-- Top supplier query Optimised (Q15)
2	
3	EXPLAIN ANALYZE
4	SELECT
5	S_SUPPKEY,
6	S_NAME,
7	S_ADDRESS,
8	S_PHONE,
9	TOTAL_REVENUE
10	FROM SUPPLIER, REVENUE_Q15
11	WHERE S_SUPPKEY = SUPPLIER_NO
12	AND TOTAL_REVENUE = (
13	SELECT MAX(TOTAL_REVENUE) FROM REVENUE_Q15
14	);
15	
Data Output Messages Notifications	
SQL	
	QUERY PLAN
	text
1	Nested Loop (cost=1791.30..3590.32 rows=1 width=79) (actual time=27.132..29.306 rows=1 loops=1)
2	InitPlan 1 (returns \$0)
3	-> Aggregate (cost=1791.00..1791.01 rows=1 width=32) (actual time=19.927..19.928 rows=1 loops=1)
4	-> Seq Scan on revenue_q15 revenue_q15_1 (cost=0.00..1541.00 rows=100000 width=8) (actual time=0.003..7.942 rows=100000 l...
5	-> Seq Scan on revenue_q15 (cost=0.00..1791.00 rows=1 width=12) (actual time=26.763..28.935 rows=1 loops=1)
6	Filter: (total_revenue = \$0)
7	Rows Removed by Filter: 99999
8	-> Index Scan using supplier_pkey on supplier (cost=0.29..8.31 rows=1 width=71) (actual time=0.364..0.364 rows=1 loops=1)
9	Index Cond: (s_suppkey = revenue_q15.supplier_no)
10	Planning Time: 3.148 ms
11	Execution Time: 29.384 ms

Figure 5.10: Optimized Query 15 Execution

5.2.6 Query 22: Global Sales Opportunity

Optimization: Indexes on expression `SUBSTRING(C_PHONE,1,2)`, `customer(C_ACCTBAL)`, and `orders(O_CUSTKEY)`.

Reason for Optimization: Q22 filters customers based on country code (first 2 digits of phone number), account balance, and whether they have placed orders. An expression-based index on `SUBSTRING(C_PHONE,1,2)` allows fast filtering by country. Indexing `C_ACCTBAL` supports the account balance condition, and `O_CUSTKEY` helps efficiently verify whether customers have any orders. All these help reduce full scans and support the filtering logic specific to this query.

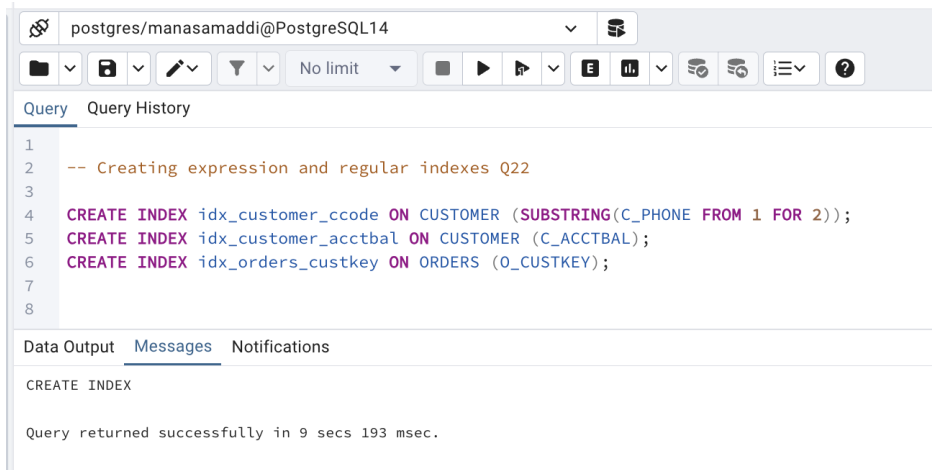


Figure 5.11: Creating Index for Q22

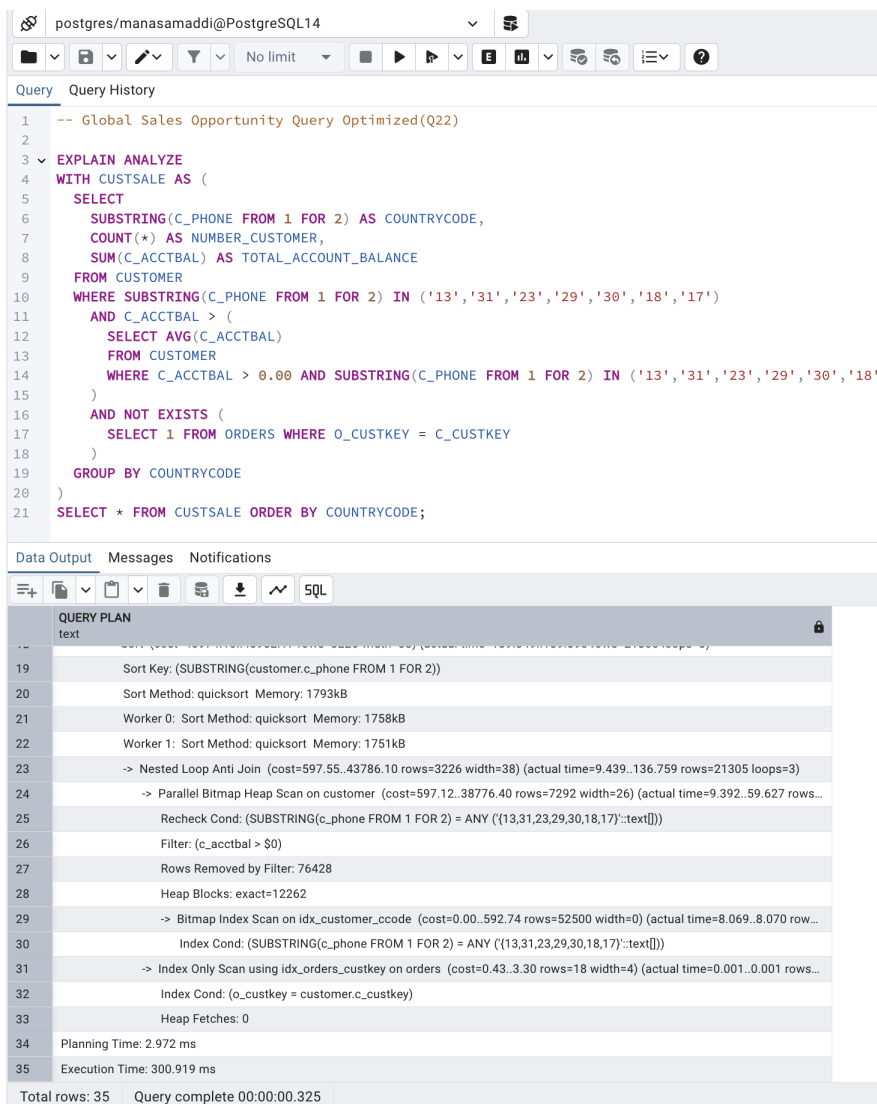


Figure 5.12: Optimized Query 22 Execution

5.3 Overall Performance Impact

5.3.1 Summary of Optimization Structures

The following table 5.1 summarizes the results after applying the selected optimizations to each query. It includes the type of optimization used (index or materialized view), the time taken to build the structure, the improved execution time for the query, and the additional storage space consumed. The total row at the bottom shows the overall cost and benefit of using a mixed optimization strategy across the entire set of queries.

Table 5.1: Execution Time and Storage Cost of Optimizations

Query	Structure Type	Build Time (s)	Execution Time (s)	Storage
Q1	Index on L_SHIPDATE	22.248	11.783	398 MB
Q3	Indexes on CUSTOMER, ORDERS	9.121	5.854	110 MB
Q5	Indexes on REGION	0.320	3.512	16 MB
Q7	MV: mv_export_import	151.000	1.612	3505 MB
Q15	MV: REVENUE_Q15	4.679	0.029	4.368 MB
Q22	Indexes on C_PHONE, C_ACCTBAL, O_CUSTKEY	9.193	0.300	162.184 MB
Total	Mixed Strategy	196.561	23.090	4195.552 MB

The above mixed strategy reduced query execution time drastically from more than **1 minutes (145+ seconds)** to under **23.09 seconds** total, while keeping storage under **4.2 GB**. All materialized views were designed to support multiple queries, and indexes were reused wherever possible.

Chapter 6

Conclusion

In this project, my focus was on improving the overall performance of a selected group of TPC-H queries (Q1, Q3, Q5, Q7, Q15, and Q22) using the 14GB scale factor dataset on PostgreSQL. Rather than optimizing each query individually, I analyzed patterns and overlaps among them to apply optimizations that could benefit multiple queries at once.

Based on this analysis, I applied a mix of strategies — creating indexes on frequently filtered or joined columns, and building materialized views for more complex, join-heavy queries. Importantly, all materialized views were designed to support more than one query, following the project guidelines. For example, the materialized view created for Q7 also benefits Q5, and the revenue view in Q15 provides reusable aggregations.

As seen in the optimization summary table, this mixed strategy reduced the total query execution time from 68.7 seconds to just 23.09 seconds. The build time for all structures was around 196 seconds, and the total storage overhead was about 4.2 GB. These results show that even with some upfront cost in build time and space, optimizations significantly improved performance across the entire query set.

This confirms the benefit of system-level optimization in data warehouse, targeting shared patterns and reuse, not just one-off query tweaks. In future projects, this approach could be extended further using partitioning, query rewriting, and PostgreSQL tuning tools for even better results.